

# Containers, Kubernetes & Helm

## In This Edition

---

|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Overview --- What It Is</b>                             | <b>4</b>  |
| <b>2</b>  | <b>Why It Exists</b>                                       | <b>4</b>  |
|           | 2.1 The deployment evolution                               | 4         |
|           | 2.2 The specific problems each layer kills                 | 5         |
| <b>3</b>  | <b>Why FAANG Cares</b>                                     | <b>5</b>  |
|           | 3.1 Google                                                 | 5         |
|           | 3.2 Meta                                                   | 5         |
|           | 3.3 Amazon (AWS)                                           | 6         |
|           | 3.4 Apple                                                  | 6         |
|           | 3.5 Netflix                                                | 6         |
| <b>4</b>  | <b>Core Concepts</b>                                       | <b>6</b>  |
| <b>5</b>  | <b>Docker &amp; Containers</b>                             | <b>7</b>  |
|           | 5.1 Containers vs VMs --- the mechanism                    | 7         |
|           | 5.2 Docker architecture: CLI → dockerd → containerd → runc | 7         |
|           | 5.3 Images & layers: union FS, copy-on-write, caching      | 8         |
|           | 5.4 Dockerfile: every key instruction                      | 9         |
|           | 5.5 CMD vs ENTRYPOINT (and exec vs shell form)             | 10        |
|           | 5.6 .dockerignore, cache invalidation, layer ordering      | 10        |
|           | 5.7 Multi-stage builds (real example)                      | 10        |
|           | 5.8 Docker networking                                      | 11        |
|           | 5.9 Docker storage: volumes vs bind mounts vs tmpfs        | 11        |
|           | 5.10 Registries                                            | 12        |
|           | 5.11 Container security                                    | 12        |
| <b>6</b>  | <b>Container Internals (Namespaces &amp; cgroups)</b>      | <b>12</b> |
|           | 6.1 Namespaces --- "what you can see"                      | 12        |
|           | 6.2 cgroups (control groups) --- "how much you can use"    | 13        |
|           | 6.3 Capabilities --- trimming root                         | 14        |
| <b>7</b>  | <b>Docker Compose</b>                                      | <b>14</b> |
| <b>8</b>  | <b>Kubernetes Architecture</b>                             | <b>15</b> |
|           | 8.1 Control plane components                               | 16        |
|           | 8.2 Worker node components                                 | 16        |
|           | 8.3 The reconciliation loop --- the soul of K8s            | 17        |
| <b>9</b>  | <b>Kubernetes Workloads</b>                                | <b>17</b> |
|           | 9.1 Pod --- the atom                                       | 17        |
|           | 9.2 ReplicaSet                                             | 18        |
|           | 9.3 Deployment --- rolling updates + rollback              | 18        |
|           | 9.4 StatefulSet --- for stateful apps                      | 19        |
|           | 9.5 DaemonSet                                              | 20        |
|           | 9.6 Job & CronJob                                          | 20        |
| <b>10</b> | <b>Kubernetes Networking</b>                               | <b>21</b> |
|           | 10.1 The Kubernetes network model (the rules)              | 21        |
|           | 10.2 CNI (Container Network Interface)                     | 21        |
|           | 10.3 Services --- stable virtual IPs                       | 21        |
|           | 10.4 kube-proxy: iptables vs IPVS                          | 22        |
|           | 10.5 CoreDNS --- service discovery                         | 22        |
|           | 10.6 Ingress + Ingress controllers                         | 22        |
|           | 10.7 NetworkPolicy                                         | 23        |
|           | 10.8 Trace: how an external request reaches a pod          | 23        |
| <b>11</b> | <b>Config, Secrets &amp; Storage</b>                       | <b>23</b> |
|           | 11.1 ConfigMap vs Secret                                   | 23        |
|           | 11.2 Consuming config: env vars vs volume mounts           | 24        |
|           | 11.3 Volumes, PV / PVC, StorageClass, CSI                  | 24        |
| <b>12</b> | <b>Scheduling &amp; Resource Management</b>                | <b>25</b> |
|           | 12.1 How the scheduler places a pod                        | 25        |
|           | 12.2 Placement controls                                    | 25        |

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
| <b>13</b> | <b>Autoscaling</b>                                        | <b>27</b> |
| 13.1      | Horizontal Pod Autoscaler (HPA)                           | 27        |
| 13.2      | Vertical Pod Autoscaler (VPA)                             | 27        |
| 13.3      | Cluster Autoscaler (CA)                                   | 28        |
| <b>14</b> | <b>Rollouts, Probes &amp; Self-Healing</b>                | <b>28</b> |
| 14.1      | The three probes (and the exact failure each prevents)    | 28        |
| 14.2      | Rolling update / rollback (recap with commands)           | 29        |
| 14.3      | Deployment strategies                                     | 29        |
| 14.4      | Self-healing --- what actually happens                    | 29        |
| <b>15</b> | <b>RBAC &amp; Security</b>                                | <b>30</b> |
| 15.1      | RBAC (Role-Based Access Control)                          | 30        |
| 15.2      | Pod / workload security                                   | 30        |
| 15.3      | Secrets at rest                                           | 31        |
| <b>16</b> | <b>Operators &amp; CRDs</b>                               | <b>31</b> |
| 16.1      | CRDs --- extending the API                                | 31        |
| 16.2      | The Operator pattern                                      | 31        |
| <b>17</b> | <b>Helm</b>                                               | <b>32</b> |
| 17.1      | The problem Helm solves: YAML sprawl                      | 32        |
| 17.2      | Chart structure                                           | 32        |
| 17.3      | Templating (Go templates)                                 | 33        |
| 17.4      | Full worked mini-chart (web app: Deployment + Service)    | 34        |
| 17.5      | Releases & revisions                                      | 35        |
| 17.6      | Values precedence                                         | 36        |
| 17.7      | Dependencies (subcharts)                                  | 36        |
| 17.8      | Essential commands                                        | 36        |
| 17.9      | Helm 2 vs Helm 3 (and why Tiller mattered)                | 37        |
| 17.10     | Helm vs Kustomize                                         | 37        |
| 17.11     | Best practices                                            | 38        |
| <b>18</b> | <b>kubectl &amp; Debugging</b>                            | <b>38</b> |
| 18.1      | kubectl essentials                                        | 38        |
| 18.2      | Debugging a broken pod --- the systematic flow            | 38        |
| <b>19</b> | <b>Production Concerns &amp; What Breaks at Scale</b>     | <b>39</b> |
| 19.1      | etcd limits                                               | 39        |
| 19.2      | Too many pods / node density                              | 40        |
| 19.3      | DNS at scale                                              | 40        |
| 19.4      | Resource starvation & noisy neighbors                     | 40        |
| 19.5      | Control-plane & rollout pitfalls                          | 40        |
| 19.6      | Multi-cluster / multi-region                              | 40        |
| <b>20</b> | <b>Architecture / Diagrams</b>                            | <b>40</b> |
| 20.1      | The full stack in one picture                             | 41        |
| 20.2      | Inside a container (the kernel view)                      | 41        |
| 20.3      | Rolling update (ReplicaSet choreography)                  | 41        |
| <b>21</b> | <b>Real-World Examples</b>                                | <b>41</b> |
| <b>22</b> | <b>Real-Life Analogies</b>                                | <b>42</b> |
| <b>23</b> | <b>Memory Tricks / Mnemonics</b>                          | <b>43</b> |
| <b>24</b> | <b>Common Interview Questions</b>                         | <b>43</b> |
| 24.1      | Q1: What's the difference between Docker and Kubernetes?  | 43        |
| 24.2      | Q2: What is a container, really? How does isolation work? | 44        |
| 24.3      | Q3: CMD vs ENTRYPOINT --- and exec vs shell form?         | 44        |
| 24.4      | Q4: Explain multi-stage builds and why they matter.       | 44        |
| 24.5      | Q5: Liveness vs readiness probe?                          | 45        |

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| 24.6      | Q6: Requests vs limits, and the QoS classes?                  | 45        |
| 24.7      | Q7: How does a rolling update work, and how do you roll back? | 45        |
| 24.8      | Q8: StatefulSet vs Deployment --- when and why?               | 46        |
| 24.9      | Q9: How does an external request reach a pod?                 | 46        |
| 24.10     | Q10: How does the scheduler decide where to place a pod?      | 46        |
| 24.11     | Q11: What problem does Helm solve, and Helm vs Kustomize?     | 47        |
| 24.12     | Q12: Walk me through debugging a CrashLoopBackOff pod.        | 47        |
| <b>25</b> | <b>Senior-Level Discussion Points</b>                         | <b>47</b> |
| 25.1      | Why Kubernetes is "just" a database of controllers            | 48        |
| 25.2      | The liveness-probe footgun and cascading failures             | 48        |
| 25.3      | Stateful workloads are the hard part                          | 48        |
| 25.4      | CPU limits considered (sometimes) harmful                     | 48        |
| 25.5      | Multi-tenancy and the shared-kernel trust boundary            | 48        |
| 25.6      | GitOps as the deployment control plane                        | 48        |
| 25.7      | When NOT to use Kubernetes                                    | 48        |
| <b>26</b> | <b>Typical Mistakes Candidates Make</b>                       | <b>49</b> |
| <b>27</b> | <b>How This Connects to Other Topics</b>                      | <b>49</b> |
| 27.1      | Distributed Systems (07-distributed-systems.md)               | 49        |
| 27.2      | Cloud & Infrastructure (04-cloud-infrastructure.md)           | 50        |
| 27.3      | System Design                                                 | 50        |
| 27.4      | Operating Systems / Linux                                     | 50        |
| 27.5      | Networking                                                    | 50        |
| 27.6      | Security                                                      | 50        |
| <b>28</b> | <b>FAANG Interview Tips</b>                                   | <b>50</b> |
| <b>29</b> | <b>Revision Cheat Sheet</b>                                   | <b>51</b> |
| 29.1      | 10-Minute Summary                                             | 51        |
| 29.2      | Key Comparison Tables (memorize)                              | 52        |
| 29.3      | Most Important Concepts (Priority Order)                      | 53        |

**How to use this file:** This is the deep-dive companion to 04-cloud-infrastructure.md. That file gives you the moderate-level map of Docker and Kubernetes; this file goes to the *mechanism* level — how isolation actually works in the Linux kernel, how the scheduler filters and scores nodes, how an external packet finds a pod, how a rolling update mutates ReplicaSets, and a full treatment of **Helm** (the biggest gap). Read it once end-to-end to build the mental model, then drill the Common Interview Questions and the Revision Cheat Sheet before each loop. Every claim here is meant to be defensible under follow-up questioning.

## 1 Overview --- What It Is

This is the **modern container deployment stack** — the layered set of technologies that takes a process running on your laptop and turns it into a fleet of identical, self-healing, auto-scaling workloads running across hundreds of machines, with zero-downtime upgrades and packaged, parameterized configuration.

Three layers, three jobs:

|                              |                                                                                         |                          |
|------------------------------|-----------------------------------------------------------------------------------------|--------------------------|
| HELM<br>(templating)         | Package + parameterize + version K8s apps<br>one chart → many environments              | ← "apt for Kubernetes"   |
| KUBERNETES<br>(orchestrator) | Orchestrate: schedule, heal, scale, network<br>declarative desired-state reconciliation | ← the control plane      |
| CONTAINERS<br>(Docker/OCI)   | Package + isolate a single process<br>namespaces + cgroups + union FS                   | ← the unit of deployment |

- ▶ A **container** is the unit: an immutable, portable bundle of app + libs + filesystem that runs as an isolated Linux process.
- ▶ **Kubernetes (K8s)** is the orchestrator: you declare *what* you want (3 replicas of this image, behind this service), and controllers continuously reconcile reality toward that desired state.
- ▶ **Helm** is the package manager: it solves "YAML sprawl" by templating Kubernetes manifests and tracking installs as versioned, rollback-able *releases*.

**One-sentence framing:** Docker answers "how do I package and isolate a process," Kubernetes answers "how do I run thousands of those reliably across many machines," and Helm answers "how do I package, parameterize, and version the Kubernetes config for one application."

## 2 Why It Exists

### 2.1 The deployment evolution

Each era solved the pain of the previous one:

|                  |                                                                                                                             |
|------------------|-----------------------------------------------------------------------------------------------------------------------------|
| BARE METAL       | One app per server. Provisioning = weeks. Utilization 5-15%.                                                                |
| ↓                | "Works on my machine." No isolation between apps.                                                                           |
| VIRTUAL MACHINES | Hypervisor slices one server into many. Better utilization, snapshot/restore. But each VM ships a full OS (GBs, slow boot). |
| ↓                | Share the host kernel; isolate with namespaces+cgroups.                                                                     |
| CONTAINERS       | MBs not GBs, ms not minutes, 100s per host. Immutable images.                                                               |
| ↓                |                                                                                                                             |

ORCHESTRATION      Containers alone don't self-heal, schedule, or load-balance.  
 ↓                      Kubernetes adds desired-state reconciliation across a fleet.  
 SERVERLESS          Push further: don't manage nodes at all (Lambda/Cloud Run /  
                              Knative). Scale-to-zero, pay-per-request. The trade: less control.

## 2.2 The specific problems each layer kills

| Pain                                                       | Solution                                                |
|------------------------------------------------------------|---------------------------------------------------------|
| "Works on my machine" — env drift between dev/staging/prod | Immutable container image: same bytes everywhere        |
| VM overhead — GB images, minute-long boots, low density    | Containers share the kernel: MB images, ms starts       |
| Manual placement — which app on which server?              | K8s scheduler bin-packs pods onto nodes automatically   |
| A crashed process stays dead                               | K8s controllers restart it; reschedule if the node dies |
| Scaling means paging an ops person                         | HPA + Cluster Autoscaler add pods/nodes on metrics      |
| Deploys are scary big-bang events                          | Rolling/canary/blue-green with automatic rollback       |
| Pod IPs change constantly                                  | Services give a stable virtual IP + DNS name            |
| Every app re-writes the same 2000 lines of YAML per env    | Helm templates one chart, parameterized by values       |

**Interview takeaway:** The evolution is *isolation getting cheaper and management getting more automated*. VMs isolate at the hardware level (own kernel, strong, heavy); containers isolate at the process level (shared kernel, weaker, cheap); orchestration automates running many of them; Helm automates configuring them per environment.

## 3 Why FAANG Cares

### 3.1 Google

- › **They invented this.** Kubernetes is the open-source descendant of **Borg** (2003) and its successor Omega. Google ran everything in containers a decade before Docker existed; they open-sourced K8s in 2014 partly to undercut AWS's lead and make the ecosystem coalesce around their model.
- › Interviews probe **K8s internals**: "walk me through the scheduler," "what's in etcd," "how does a controller reconcile," "Borg vs K8s design decisions," and SRE concepts (SLO/error budget — Google coined them).

### 3.2 Meta

- › Runs one of the largest fleets on earth via **Twine/Tupperware** (internal, K8s-like). Cares about resource efficiency and deployment velocity at tens of thousands of services.
- › Focus: large-scale scheduling trade-offs, why you might build custom orchestration, bin-packing efficiency.

### 3.3 Amazon (AWS)

- › Containers are core revenue: **ECS, EKS, Fargate, ECR, App Runner**. Engineers are expected to know the difference between ECS and EKS and when Fargate (no nodes to manage) beats self-managed nodes.
- › Focus: operational excellence (first Well-Architected pillar), cost optimization, IaC integration.

### 3.4 Apple

- › Large on-prem + private cloud; growing internal K8s. Prioritizes **security and privacy** — expect pod security, secrets-at-rest, supply-chain (image signing) questions.

### 3.5 Netflix

- › Pioneered **chaos engineering**, microservices on AWS, canary analysis (Kayenta/Spinnaker). Their Titus is a container platform predating wide EKS use.
- › Focus: progressive delivery, automated canary analysis, resilience.

**Interview takeaway:** K8s = Google's gift from Borg. Chaos engineering + automated canary = Netflix. Managed container services (ECS/EKS/Fargate) = Amazon. Tie your answers to the company: at Amazon talk Fargate/cost, at Google talk scheduler/etcd internals.

## 4 Core Concepts

A vocabulary spine before the deep dives — every term below gets a full section later.

| Concept                   | One-liner                                                                 |
|---------------------------|---------------------------------------------------------------------------|
| <b>Image</b>              | Immutable, layered filesystem + metadata; the build artifact              |
| <b>Container</b>          | A running (or stopped) instance of an image = isolated Linux process      |
| <b>OCI</b>                | Open Container Initiative: the image + runtime standards everyone follows |
| <b>Registry</b>           | Where images live (Docker Hub, ECR, GCR); push/pull by tag or digest      |
| <b>Pod</b>                | Smallest K8s deployable unit; 1+ containers sharing net/IPC/volumes       |
| <b>ReplicaSet</b>         | Keeps N identical pods running                                            |
| <b>Deployment</b>         | Manages ReplicaSets to do rolling updates + rollback                      |
| <b>StatefulSet</b>        | Like Deployment but stable identity + ordering + per-pod storage          |
| <b>Service</b>            | Stable virtual IP + DNS in front of a changing set of pods                |
| <b>Ingress</b>            | L7 (HTTP) router into the cluster (host/path/TLS)                         |
| <b>ConfigMap / Secret</b> | Externalized non-secret / sensitive config                                |
| <b>PV / PVC</b>           | Cluster storage resource / a pod's claim on it                            |
| <b>Controller</b>         | A loop reconciling actual state → desired state                           |
| <b>CRD / Operator</b>     | Custom resource type + a controller that manages it                       |
| <b>Helm Chart</b>         | A templated, versioned package of K8s manifests                           |
| <b>Helm Release</b>       | A specific install of a chart, with revision history                      |

The single most important idea, threaded through everything:

**Declarative reconciliation.** You never tell Kubernetes "start a container." You declare desired state ("I want 3 replicas of image X"). Controllers run an infinite loop: observe actual state, diff against desired, take corrective action, repeat. This is *why* K8s self-heals

– there's no special "recovery" code path; healing is just the normal reconciliation loop noticing a difference (2 pods running, 3 desired) and fixing it.

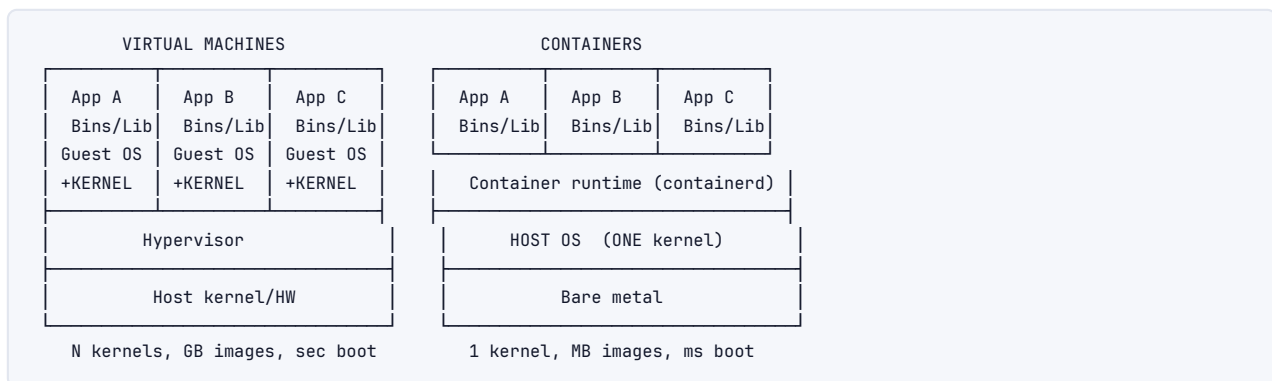
**Interview takeaway:** If you can articulate "declarative desired state + reconciliation loops" clearly, you've shown you understand the *soul* of Kubernetes, not just the API surface.

## 5 Docker & Containers

### 5.1 Containers vs VMs --- the mechanism

A **VM** virtualizes *hardware*: a hypervisor (Type 1 like KVM/ESXi on bare metal, Type 2 like VirtualBox on an OS) presents virtual CPUs, virtual NICs, and virtual disks. Each VM boots its **own kernel** and OS. Isolation is strong because the boundary is the virtual hardware itself.

A **container** virtualizes the *operating system*: it's just a normal Linux process on the host, but the kernel has been told (via **namespaces**) to give that process a private view of PIDs, network, mounts, etc., and (via **cgroups**) to cap its CPU/memory. There is **no guest kernel** – every container shares the host's one kernel.



| Dimension          | Containers                             | VMs                                    |
|--------------------|----------------------------------------|----------------------------------------|
| Isolation boundary | Kernel namespaces + cgroups            | Virtual hardware (hypervisor)          |
| Kernel             | Shared with host                       | Own kernel per VM                      |
| Boot               | Milliseconds                           | Seconds–minutes                        |
| Image size         | MBs                                    | GBs                                    |
| Density / host     | 100s                                   | 10s                                    |
| Security isolation | Weaker (shared kernel = escape risk)   | Stronger (full HW isolation)           |
| Best for           | Microservices, CI, stateless scale-out | Multi-tenant, legacy, strong isolation |

**The hardening middle ground:** When you need VM-grade isolation with container UX, use **gVisor** (runsc – a user-space kernel that intercepts syscalls) or **Kata Containers** (a micro-VM per container). FAANG multi-tenant platforms (e.g., serverless backends) lean on these because a shared kernel + hostile tenants = container escape risk.

### 5.2 Docker architecture: CLI → dockerd → containerd → runc

When you type `docker run nginx`, four components hand off:



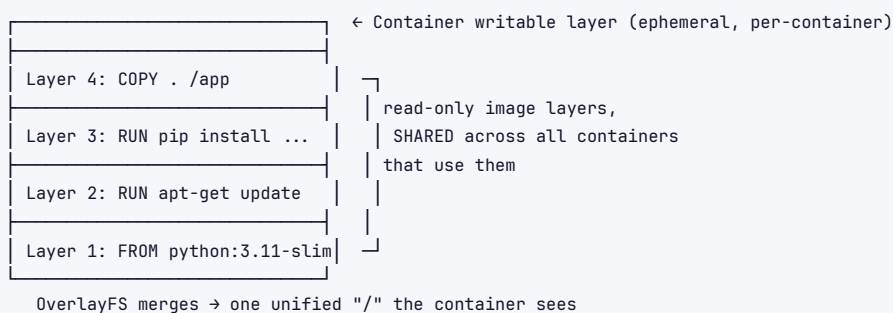
- › **runc exits after setup.** It's not a long-running parent. A small containerd-shim stays as the container's parent so the container survives a dockerd restart and so exit codes/stdout are captured. This is why you can restart the Docker daemon without killing running containers.
- › **Kubernetes skips dockerd entirely** since v1.24 (dockershim removed): the kubelet talks **CRI** directly to **containerd** (or CRI-O), which talks to runc. Docker the daemon is a developer tool; the runtime underneath is the same OCI stack.

**OCI (Open Container Initiative)** defines two specs so nothing is locked to Docker:

- › **Image spec:** layered tarballs + a JSON config + a manifest (what a `.tar` image *is*).
- › **Runtime spec:** a `config.json` bundle describing namespaces, cgroups, mounts (what a runtime must set up). Any OCI runtime (runc, crun, gVisor's runsc, Kata) can run any OCI image.

### 5.3 Images & layers: union FS, copy-on-write, caching

An image is an **ordered stack of read-only layers**, each one the filesystem diff produced by a Dockerfile instruction — conceptually like git commits. At runtime, the layers are merged into a single view by a **union filesystem** (OverlayFS today; AUFS/devicemapper historically), and a thin **writable layer** is added on top for the running container.



- › **lowerdir** = the read-only image layers (stacked); **upperdir** = the container's writable layer; **merged** = what the container sees.
- › **Copy-on-write (CoW):** read a file from a lower layer → served directly. *Write* it → OverlayFS first copies it up to the writable layer, then modifies the copy. The original layer is untouched, so other containers sharing it are unaffected. This is why 50 containers off a 500 MB base cost ~500 MB + 50 small diffs, not 25 GB.
- › **Layer caching at build time:** Docker hashes each instruction + its inputs. If nothing changed, it reuses the cached layer. The first instruction whose inputs change **busts the cache for itself and everything after it** — ordering is everything (see Dockerfile section).

**Image vs container:** image = the class (immutable template on disk); container = an instance (image layers + a writable layer + a running process). Stop a container and the writable

layer persists until you `rm` it; that's why data written inside a container (not to a volume) is lost on `docker rm`.

### Tags vs digests:

- ▶ A **tag** (`nginx:1.25`) is a *mutable pointer* — the maintainer can re-push 1.25 to point at different bytes. `latest` is just a tag with no special meaning except "the default."
- ▶ A **digest** (`nginx@sha256:abc123...`) is the *content hash of the manifest* — immutable, reproducible. In production, **pin by digest** so a re-tag upstream can't silently change what you deploy. Tags are for humans; digests are for machines and reproducibility.

## 5.4 Dockerfile: every key instruction

```

1 # Pin a specific, minimal base by tag (digest-pin in prod). Sets the base layer.
2 FROM python:3.11.7-slim
3
4 # ARG = build-time variable (not in final image env). Can have a default.
5 ARG APP_VERSION=0.0.0
6 # ENV = runtime environment variable (persists into running container).
7 ENV PYTHONUNBUFFERED=1 APP_VERSION=${APP_VERSION}
8
9 # Set the working directory (created if missing); affects subsequent RUN/CMD/COPY.
10 WORKDIR /app
11
12 # COPY deps FIRST so this layer caches unless requirements change.
13 COPY requirements.txt .
14 # Chain RUN with && and clean apt lists in the SAME layer (else cruft persists).
15 RUN pip install --no-cache-dir -r requirements.txt
16
17 # COPY the app LAST — it changes every build, so it busts cache least.
18 COPY . .
19
20 # Documents the port (does NOT publish it; -p / Service does the actual mapping).
21 EXPOSE 8080
22
23 # Declares a mount point; data here bypasses the union FS / writable layer.
24 VOLUME ["/data"]
25
26 # Run as a non-root user for least privilege (create it first via RUN).
27 RUN useradd -u 10001 -m appuser
28 USER appuser
29
30 # Container-level health signal (Docker marks healthy/unhealthy).
31 HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
32   CMD curl -fsS http://localhost:8080/health || exit 1
33
34 # ENTRYPOINT = the fixed executable; CMD = default args (overridable on `docker run`).
35 ENTRYPOINT ["python", "-m", "myapp"]
36 CMD ["--port", "8080"]

```

### Key instruction notes:

- ▶ **COPY vs ADD:** Prefer `COPY` (just copies files). `ADD` additionally auto-extracts local tarballs and can fetch URLs — surprising and a supply-chain risk. Use `ADD` only for the tar-extract feature.
- ▶ **EXPOSE** is documentation/metadata only. Actual exposure is `docker run -p 8080:8080` or a K8s Service.
- ▶ **VOLUME** declares a path that should live outside the image's CoW layer (persistence + perf). Anonymous volumes get auto-created; surprising in CI, so often managed explicitly instead.
- ▶ **USER:** containers run as **root by default**. Root in the container is root on the host kernel.

(unless user-namespace remapping is on) → a container escape = host root. Always drop to a non-root UID.

## 5.5 CMD vs ENTRYPOINT (and exec vs shell form)

This is a guaranteed interview question. Two orthogonal axes:

**1) Roles:** ENTRYPOINT = the executable that always runs. CMD = default arguments (or default command if no ENTRYPOINT). At `docker run`, anything you append on the command line **replaces CMD** but is *appended as args to ENTRYPOINT*.



```

1 ENTRYPOINT ["python", "-m", "myapp"]
2 CMD ["--port", "8080"]

```

- › `docker run img → python -m myapp --port 8080`
- › `docker run img --port 9090 → python -m myapp --port 9090` (CMD replaced, ENTRYPOINT kept)

If you used only CMD `["python", "-m", "myapp", "--port", "8080"]`, then `docker run img bash` would run `bash` instead — the whole command is replaceable. **Use ENTRYPOINT to fix the binary; use CMD for tweakable defaults.**

### 2) Form — exec vs shell:

- › **Exec form** (JSON array): `ENTRYPOINT ["python", "app.py"]` → runs the binary **directly as PID 1**. No shell, so **signals (SIGTERM) reach your process** → graceful shutdown works. Preferred.
- › **Shell form** (string): `ENTRYPOINT python app.py` → Docker wraps it as `/bin/sh -c "python app.py"`. Now **sh is PID 1** and your app is a child; SIGTERM goes to sh, which doesn't forward it → K8s kills you with SIGKILL after the grace period, dropping in-flight requests. Also no variable expansion benefits worth the cost.

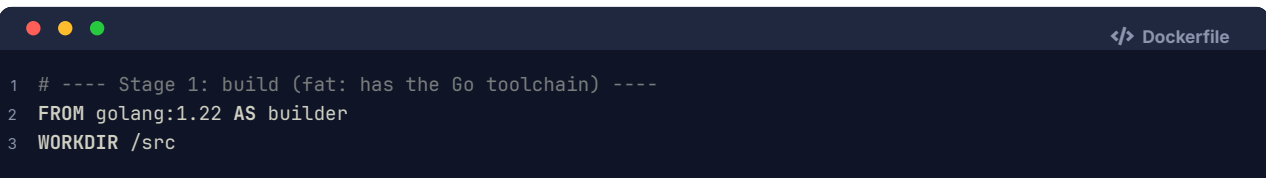
**Interview takeaway:** Exec form + ENTRYPOINT-for-binary/CMD-for-args is the correct default. The shell-form gotcha (PID 1 doesn't forward signals → ungraceful shutdown) is the senior-level detail; the fix is exec form, or a tiny init like `tini` or `--init` to reap zombies and forward signals.

## 5.6 .dockerignore, cache invalidation, layer ordering

- › **.dockerignore** excludes paths from the *build context* sent to the daemon (`.git`, `node_modules`, `*.log`, `Dockerfile`, `secrets`). Without it, `COPY . .` ships junk into layers, bloats images, leaks secrets, and breaks caching (a changed `.git` busts the COPY layer every commit).
- › **Cache invalidation rule:** order instructions **least-frequently-changed first**. Dependencies (rarely change) before app code (changes every commit). `COPY requirements.txt && pip install` before `COPY . .` means dependency reinstall only happens when deps actually change — turning a 2-minute build into a 5-second one.

## 5.7 Multi-stage builds (real example)

The problem: build tools (compilers, dev headers, `npm`, full SDKs) bloat the final image and widen the attack surface. **Multi-stage** builds in a fat stage, then copies *only the artifact* into a tiny runtime stage.



```

1 # --- Stage 1: build (fat: has the Go toolchain) ---
2 FROM golang:1.22 AS builder
3 WORKDIR /src

```

```

4 COPY go.mod go.sum ./
5 RUN go mod download          # cached unless go.mod/go.sum change
6 COPY . .
7 # Static binary, no libc dependency → can run on a scratch/distroless base.
8 RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /app/server ./cmd/server
9
10 # ---- Stage 2: runtime (tiny: just the binary) ----
11 FROM gcr.io/distroless/static-debian12:nonroot
12 COPY --from=builder /app/server /server
13 USER nonroot:nonroot
14 EXPOSE 8080
15 ENTRYPOINT ["/server"]

```

- › Final image ≈ the size of the binary (single-digit MB) instead of ~1 GB with the Go toolchain.
- › `--from=builder` copies across stages; only the last stage ships.
- › **distroless** images contain no shell, no package manager, no `ls` — drastically smaller attack surface (an attacker who lands code can't `curl | sh`). The trade: debugging is harder (no shell) — use `kubectl debug` ephemeral containers. **alpine** (musl libc, ~5 MB, has a shell via `/bin/sh`) is the middle ground; watch for musl-vs-glibc compatibility bugs and DNS quirks.

**Interview takeaway:** Multi-stage builds = “build fat, ship thin.” Smaller image → faster pulls (faster scaling/rollouts), lower cost, smaller CVE surface. Pair with distroless + non-root for the senior answer.

## 5.8 Docker networking

| Mode                       | Mechanism                                                                         | Use case                                             |
|----------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------|
| <b>bridge</b><br>(default) | Linux bridge <code>docker0</code> ; containers get 172.17.0.0/16 IPs; NAT to host | Single-host dev; containers talk via the bridge      |
| <b>host</b>                | Container shares the host's network namespace (no isolation)                      | Max network perf; container's port 80 = host port 80 |
| <b>none</b>                | Loopback only, no external networking                                             | Air-gapped batch jobs                                |
| <b>overlay</b>             | VXLAN tunnel across hosts (Swarm/K8s CNI builds on this idea)                     | Multi-host clusters                                  |
| <b>macvlan</b>             | Container gets a MAC on the physical LAN                                          | Appliance-like, legacy integration                   |

- › **Port publishing:** `-p 8080:80` adds an iptables DNAT rule mapping host:8080 → container:80.
- › **Embedded DNS:** on a user-defined bridge network, Docker runs a DNS resolver at 127.0.0.11 so containers reach each other **by name** (`db`, `web`) instead of brittle IPs. (The default bridge network lacks this — another reason to create your own.)

## 5.9 Docker storage: volumes vs bind mounts vs tmpfs

| Type              | What it is                                                                                  | Lifecycle                | Use case                          |
|-------------------|---------------------------------------------------------------------------------------------|--------------------------|-----------------------------------|
| <b>Volume</b>     | Docker-managed dir under <code>/var/lib/docker/volumes</code> ; drivers (local, NFS, cloud) | Independent of container | Production persistence (DB data)  |
| <b>Bind mount</b> | A specific host path mounted in                                                             | Host owns it             | Dev (mount source for hot reload) |

| Type         | What it is                 | Lifecycle           | Use case                                 |
|--------------|----------------------------|---------------------|------------------------------------------|
| <b>tmpfs</b> | In-memory, never hits disk | Dies with container | Secrets/scratch you don't want persisted |

The container's writable layer is **ephemeral** — `docker rm` deletes it. Anything that must survive (database files) goes in a **volume**. Bind mounts couple you to host layout (less portable); volumes are the production default.

## 5.10 Registries

- › **Where images live:** Docker Hub (public default), AWS **ECR**, Google **GCR/Artifact Registry**, Azure ACR, self-hosted Harbor.
- › **Flow:** `docker build -t myrepo/app:1.2 . → docker push myrepo/app:1.2 → cluster nodes pull` on first use (then cache by digest).
- › **Pin by digest in prod**, as above. Use **pull-through cache** / `imagePullPolicy: IfNotPresent` to avoid re-pulling on every scale-up.

## 5.11 Container security

- › **Image scanning:** Trivy/Grype/Snyk scan layers for known CVEs in OS packages and language deps. Gate CI on it.
- › **Least privilege:** non-root USER, `--read-only root FS`, drop Linux capabilities (`--cap-drop=ALL --cap-add=NET_BIND_SERVICE`), `--security-opt no-new-privileges`.
- › **Rootless mode:** run dockerd/podman as an unprivileged user (uses user namespaces) so a daemon compromise isn't host root.
- › **Signing & provenance:** **Sigstore/cosign** signs images; **SBOM** (Software Bill of Materials) records contents; admission controllers reject unsigned images. This is the supply-chain story FAANG (esp. Apple/Google) cares about.

**Interview takeaway:** "Minimal base (distroless/alpine) + non-root + scanned + signed + digest-pinned + read-only FS + dropped capabilities" is the security checklist a senior recites.

# 6 Container Internals (Namespaces & cgroups)

The single most important mental model: **a container is just a Linux process** with two kernel features layered on. There is no "container" object in the kernel — there's a process whose namespaces give it a private *view* and whose cgroups *cap* its resources. Plus capabilities to trim root's powers.

```
A "container" = a process tree where:
├─ NAMESPACES decide WHAT IT CAN SEE (isolation / visibility)
├─ CGROUPS     decide HOW MUCH IT CAN USE (limits / accounting)
└─ CAPABILITIES decide WHAT ROOT CAN DO (privilege trimming)
```

## 6.1 Namespaces --- "what you can see"

Namespaces partition kernel resources so a process gets its own isolated instance. Created via `clone()/unshare()/setns()` syscalls (which `runc` invokes).

| Namespace  | Isolates    | Effect inside the container                                  |
|------------|-------------|--------------------------------------------------------------|
| <b>pid</b> | Process IDs | Your main process is <b>PID 1</b> ; can't see host processes |

| Namespace          | Isolates                            | Effect inside the container                                    |
|--------------------|-------------------------------------|----------------------------------------------------------------|
| <b>net</b>         | NICs, routes, iptables, ports       | Own eth0, own loopback, own port space                         |
| <b>mnt</b>         | Mount points                        | Own root filesystem / (the image rootfs)                       |
| <b>uts</b>         | Hostname / domain                   | hostname returns the pod/container name                        |
| <b>ipc</b>         | SysV IPC, shared memory, semaphores | Can't see host's shared-memory segments                        |
| <b>user</b>        | UID/GID mapping                     | UID 0 <i>inside</i> can map to unprivileged UID <i>outside</i> |
| <b>cgroup</b>      | cgroup root view                    | Sees its own cgroup tree, not the host's                       |
| <b>time</b> (5.6+) | CLOCK_MONOTONIC/BOOTTIME offset     | Different boot/clock offset                                    |

Verify on any Linux box: `ls -l /proc/self/ns/` shows your current namespaces; two processes sharing a namespace have the same inode there.

**The user namespace is the security crown jewel:** with it, root (UID 0) *inside* the container maps to an unprivileged UID *on the host*. So even a full container compromise as "root" isn't host root. Many setups don't enable it by default (compat cost), which is why "root in container = root on host" remains the default threat.

**Pods and shared namespaces:** all containers in a K8s pod share the **net** namespace (same IP, talk via localhost) and **ipc**, but each has its own **mnt** (own filesystem) and **pid** (unless shareProcessNamespace). That shared net namespace is provided by the **pause container** (below).

## 6.2 cgroups (control groups) --- "how much you can use"

cgroups limit and *account* resource usage so one container can't starve the host. cgroup **v2** (unified hierarchy, now default) replaced the v1 per-controller mess.

| Controller    | Knob                                        | Effect                                                        |
|---------------|---------------------------------------------|---------------------------------------------------------------|
| <b>cpu</b>    | cpu.weight (shares), cpu.max (quota/period) | Relative scheduling weight + a hard ceiling via CFS bandwidth |
| <b>memory</b> | memory.max, memory.high                     | Hard limit (OOM at max) + soft throttle (reclaim at high)     |
| <b>io</b>     | io.max, io.weight                           | Block-device bandwidth/IOPS caps                              |
| <b>pids</b>   | pids.max                                    | Cap process count → stops fork bombs                          |

### The two failure modes you MUST distinguish:

- › **CPU over limit → throttling.** CPU is *compressible*. The CFS scheduler simply gives the cgroup no more CPU slices this period; the process **slows down** (latency spikes, p99 climbs) but keeps running. You'll see `nr_throttled` rising. Nobody dies.
- › **Memory over limit → OOMKill.** Memory is *incompressible* — you can't "slow down" RAM usage. When a cgroup hits `memory.max`, the kernel **OOM-kills** a process in it (exit code 137 = 128+SIGKILL). In K8s the container restarts (CrashLoop if it keeps happening).

**Interview takeaway:** "Namespaces = what you can see, cgroups = what you can use." And the killer follow-up answer: **CPU limit throttles (compressible), memory limit OOM-kills (incompressible)** — which is why teams often set memory limits but are cautious with CPU limits (throttling causes mysterious latency).

## 6.3 Capabilities --- trimming root

Linux split root's omnipotence into ~40 **capabilities** (CAP\_NET\_BIND\_SERVICE to bind <1024, CAP\_NET\_ADMIN, CAP\_SYS\_ADMIN — the "new root," CAP\_CHOWN, ...). Best practice: drop ALL, then add back only what's needed. A web server that binds port 80 needs only NET\_BIND\_SERVICE, not full root. This shrinks the blast radius of a compromise.

```
1 securityContext:
2   runAsNonRoot: true
3   runAsUser: 10001
4   allowPrivilegeEscalation: false
5   readOnlyRootFilesystem: true
6   capabilities:
7     drop: ["ALL"]
8     add: ["NET_BIND_SERVICE"]
```

**Putting it together:** when runc starts a container it (1) clone()s with the namespace flags, (2) writes the cgroup limits, (3) pivots root to the image rootfs, (4) drops capabilities + applies seccomp/AppArmor, then (5) exec()s your process. That process *is* the container.

## 7 Docker Compose

**Compose** defines and runs **multi-container apps on a single host** via one declarative YAML. It's the local-dev/integration-test counterpart to Kubernetes (Compose is not an orchestrator across machines — that's K8s).

```
1 # docker-compose.yml - a web app + Postgres + Redis
2 services:
3   web:
4     build: . # build from local Dockerfile
5     image: myapp:dev
6     ports:
7       - "8080:8080" # host:container
8     environment:
9       DATABASE_URL: postgres://app:secret@db:5432/app
10      REDIS_URL: redis://cache:6379
11     depends_on:
12       db:
13         condition: service_healthy # wait for db's healthcheck, not just "started"
14       cache:
15         condition: service_started
16     networks: [backend]
17
18   db:
19     image: postgres:16
20     environment:
21       POSTGRES_USER: app
22       POSTGRES_PASSWORD: secret
23       POSTGRES_DB: app
24     volumes:
25       - db-data:/var/lib/postgresql/data # named volume → survives `compose down`
26     healthcheck:
27       test: ["CMD-SHELL", "pg_isready -U app"]
28       interval: 5s
29       timeout: 3s
```

```

30     retries: 5
31     networks: [backend]
32
33     cache:
34         image: redis:7-alpine
35         networks: [backend]
36
37     volumes:
38         db-data:                # declared named volume
39
40     networks:
41         backend:                # user-defined network → DNS by service name

```

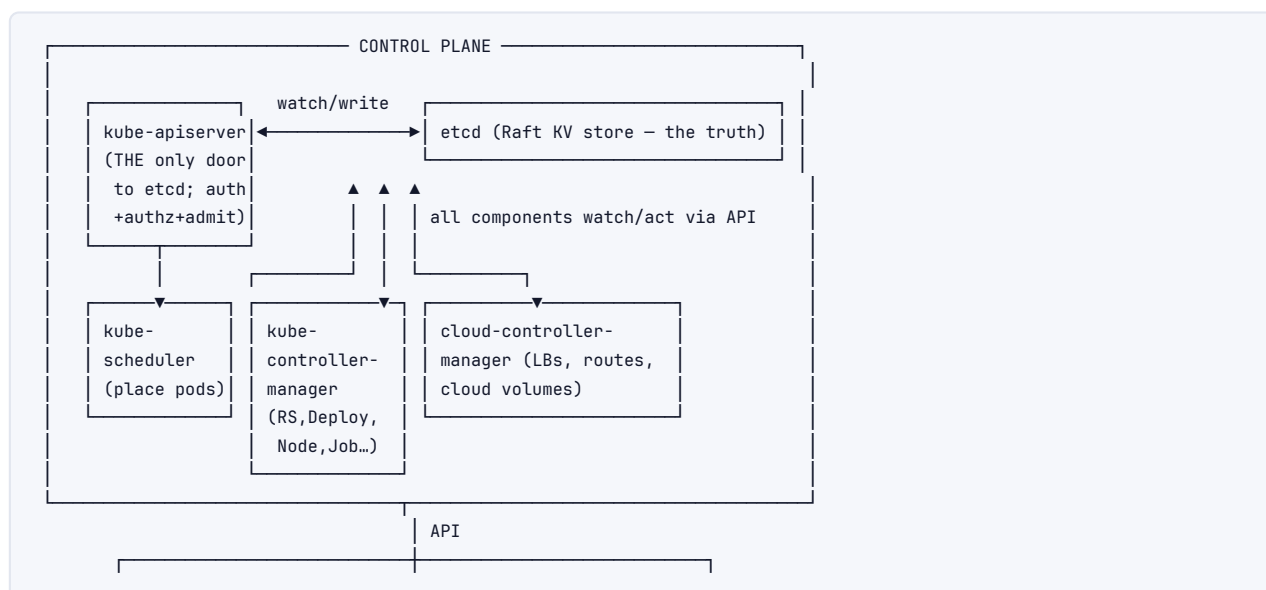
Mechanics worth knowing:

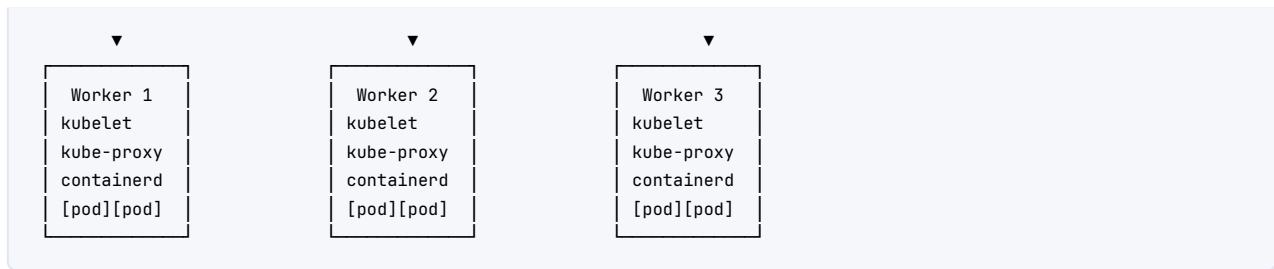
- ▶ **Service-name DNS:** Compose creates a user-defined network; web reaches Postgres at host db and Redis at cache — the service names resolve via Docker's embedded DNS. No IPs anywhere.
- ▶ **depends\_on controls start order, not readiness** by default — the dependency container being *started* ≠ *ready to accept connections*. Use condition: `service_healthy` (tied to the dependency's healthcheck) for true readiness, or your app must retry its connection.
- ▶ **Named volumes** (db-data) persist across docker compose down; only `down -v` wipes them.
- ▶ `docker compose up -d` (background), `logs -f`, `ps`, `down`. **Compose ≠ production orchestration** — no multi-node scheduling, no self-healing across machines, no rolling updates. It's for dev/CI; you graduate to Kubernetes (often via kompose to convert, then refine).

**Interview takeaway:** Compose = multi-container *single-host* dev/test convenience with DNS-by-service-name. The `depends_on` start-order vs readiness distinction is the gotcha; the answer is healthchecks + app-side retries.

## 8 Kubernetes Architecture

Kubernetes is a **control plane** (the brain — makes global decisions, stores state) plus **worker nodes** (the muscle — run the actual pods). Everything funnels through one API server; everything is driven by reconciliation loops.





## 8.1 Control plane components

**kube-apiserver** — the *only* component that talks to etcd, and the single front door for everything (kubectl, controllers, kubelets, the dashboard). It (1) authenticates, (2) authorizes (RBAC), (3) runs **admission controllers** (mutating then validating webhooks — e.g., inject sidecars, enforce policy), (4) validates the object schema, (5) persists to etcd. Stateless → run 3+ replicas behind a load balancer for HA.

**etcd** — the cluster's source of truth: a distributed, strongly-consistent key-value store using **Raft** (writes need a majority quorum; see 07-distributed-systems.md). Stores *every* object: /registry/pods/default/web-abc. Lose etcd = lose the cluster's brain → **etcd backups are the #1 operational duty**. Only the API server touches it. It's CP: during a partition the minority side can't accept writes.

**kube-scheduler** — watches for pods with no nodeName and assigns each to a node in two phases: **filter** (which nodes *can* run this?) then **score** (which is *best*?). Writes the chosen nodeName back via the API server (it doesn't start the pod — the kubelet on that node does). Full detail in [Scheduling](#).

**kube-controller-manager** — one process running *many* independent control loops. Each loop watches desired vs actual and reconciles:

| Controller                 | Reconciles                                                   |
|----------------------------|--------------------------------------------------------------|
| ReplicaSet                 | N pods exist for a given template                            |
| Deployment                 | The right ReplicaSets exist for a rollout                    |
| Node                       | Marks NotReady nodes, evicts their pods after a grace period |
| Job/CronJob                | Pods run to completion / on schedule                         |
| Endpoints / EndpointSlice  | Which pod IPs back each Service                              |
| ServiceAccount / Namespace | Defaults + cleanup                                           |

**cloud-controller-manager** — isolates cloud-specific logic: provisions a real load balancer for Service type=LoadBalancer, sets up node routes, attaches cloud disks. This split lets the core be cloud-agnostic.

## 8.2 Worker node components

**kubelet** — the node agent. Watches the API server for pods assigned to *its* node, then drives the container runtime (over **CRI**) to pull images and start containers. Runs probes, restarts failed containers, reports node conditions (MemoryPressure, DiskPressure, PIDPressure) and pod status back up. The kubelet is *not* a controller in the kube-controller-manager — it's the per-node executor.

**kube-proxy** — implements the **Service** abstraction on each node. In **iptables** mode it installs DNAT rules so a packet to a ClusterIP is rewritten to a random backing pod IP — *the kernel does the routing; kube-proxy doesn't sit in the data path*. **IPVS** mode (hash tables) scales to 10k+ services with O(1) lookup vs iptables' O(n) rule walk. eBPF dataplanes (Cilium) can replace kube-proxy entirely.

**Container runtime (via CRI)** — containerd (default) or CRI-O; the kubelet speaks the **Container Runtime Interface** so any conformant runtime works. dockershim was removed in 1.24.

### 8.3 The reconciliation loop --- the soul of K8s

```

CONTROLLER LOOP (forever)

desired = read spec from API server (etcd)
actual  = observe real world (pods, nodes...)

if actual ≠ desired:
    take ONE corrective action toward desired
    (create/delete a pod, scale an RS, etc.)
    write status back to API server

... watch for changes, repeat ...

```

You `kubectl apply` a Deployment wanting 3 replicas → stored in etcd. Deployment controller creates a ReplicaSet → ReplicaSet controller sees "0 pods, want 3" → creates 3 Pod objects → scheduler assigns nodes → kubelets start containers. A node dies: Node controller marks pods gone → ReplicaSet sees "2 pods, want 3" → creates one more → scheduler places it elsewhere. **No special recovery code — healing is just the loop noticing a diff.** This is *level-triggered* (converge to a state), not *edge-triggered* (react to events), which is why K8s is robust to missed events and restarts.

**Interview takeaway:** Name the control-plane four (apiserver, etcd, scheduler, controller-manager) and the node three (kubelet, kube-proxy, runtime), then nail the punchline: *everything is controllers running level-triggered reconciliation loops through the one API server; etcd is the single source of truth.*

## 9 Kubernetes Workloads

### 9.1 Pod --- the atom

The smallest deployable unit: one or more containers that **share the network namespace** (one IP, reach each other via `localhost`), share **IPC**, and can share **volumes**. Pods are *ephemeral and disposable* — you almost never create them directly; a controller does, so a replacement appears when one dies (with a *new* IP and name).

**The pause container:** every pod has a hidden "pause" (infra) container that holds the network namespace open. App containers join *its* namespace. So if your app container crashes and restarts, the **pod keeps its IP** because the pause container — and thus the netns — never went away. The pause container does nothing but `sleep` and `reap`.

**Init containers** run *to completion, in order, before* app containers start. Use for setup that must finish first: wait for a dependency, run a DB migration, fetch config, set kernel sysctls. If an init container fails, the pod restarts it (per `restartPolicy`) and app containers never start.

**Sidecar pattern:** a helper container co-located in the pod to handle a cross-cutting concern, sharing the pod's network/volumes: a logging shipper, a metrics exporter, a service-mesh proxy (Envoy/Istio), a config reloader. Kubernetes 1.28+ added *native sidecars* (an init container with `restartPolicy: Always`) that start before and stop after the main container — fixing the old race where the app outlived/under-lived its proxy.

```

1  apiVersion: v1
2  kind: Pod
3  metadata:
4    name: web
5  spec:
6    initContainers:
7      - name: wait-for-db
8        image: busybox:1.36
9        command: ['sh', '-c', 'until nc -z db 5432; do echo waiting; sleep 2; done']
10   containers:
11     - name: app                                # main
12       image: myapp:1.4
13       ports: [{containerPort: 8080}]
14       volumeMounts: [{name: logs, mountPath: /var/log/app}]
15     - name: log-shipper                        # sidecar
16       image: fluent-bit:2.2
17       volumeMounts: [{name: logs, mountPath: /var/log/app, readOnly: true}]
18   volumes:
19     - name: logs
20       emptyDir: {}                          # shared scratch between the two containers

```

**Pod lifecycle phases:** Pending (accepted, not yet running — scheduling or image pull) → Running → Succeeded/Failed (terminal) ; plus Unknown. Container states: Waiting (e.g., ImagePullBackOff), Running, Terminated.

## 9.2 ReplicaSet

Keeps exactly N pods matching a label selector alive. You rarely touch it directly — Deployments own it. Its loop: count pods matching the selector; create/delete to reach replicas.

## 9.3 Deployment --- rolling updates + rollback

The workhorse for **stateless** apps. A Deployment manages *ReplicaSets* to achieve zero-downtime upgrades. The key mechanism: a new image creates a **new ReplicaSet**; the Deployment scales the new RS up and the old RS down gradually, governed by `maxSurge`/`maxUnavailable`.

```

1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: web
5  spec:
6    replicas: 4
7    strategy:
8      type: RollingUpdate
9      rollingUpdate:
10        maxSurge: 1          # may run 1 ABOVE desired during the roll (5 total)
11        maxUnavailable: 0    # never drop BELOW desired (true zero-downtime)
12    selector:
13      matchLabels: {app: web}
14    template:
15      metadata:
16        labels: {app: web}
17      spec:
18        containers:
19          - name: web
20            image: myapp:1.5    # change this → triggers a rollout
21            readinessProbe:

```

```
22 httpGet: {path: /ready, port: 8080}
```

Rolling-update mechanics (maxSurge=1, maxUnavailable=0, 4 replicas):

```
[v1][v1][v1][v1]      start
[v1][v1][v1][v1][v2]    surge 1 new (5 running); wait until v2 READY
[v1][v1][v1][v2]        new ready → kill 1 old (back to 4)
[v1][v1][v1][v2][v2]    surge another...
... repeat until ...
[v2][v2][v2][v2]        done; old RS scaled to 0 (kept for rollback)
```

- ▶ **Readiness gates the roll:** a new pod counts as "available" only when its readiness probe passes, so traffic never hits a not-ready pod and the roll won't proceed past a broken new version (it stalls instead of nuking the old ones).
- ▶ **Rollback:** `kubectl rollout undo deploy/web` scales the *previous* ReplicaSet back up and the current one down — instant, because the old RS (and its pod template) was retained (`revisionHistoryLimit`). `kubectl rollout status / rollout history` track it.
- ▶ **maxSurge vs maxUnavailable** = the speed/capacity dial. `maxUnavailable: 0` = strict zero-downtime but needs spare capacity for the surge pod; higher `maxUnavailable` rolls faster but temporarily reduces capacity.

## 9.4 StatefulSet --- for stateful apps

For databases, Kafka, ZooKeeper, anything needing **stable identity and storage**. Differs from Deployment in four ways:

|                  | Deployment                   | StatefulSet                                                                |
|------------------|------------------------------|----------------------------------------------------------------------------|
| Pod names        | random hash<br>(web-7d9-x2k) | stable ordinal (db-0, db-1, db-2)                                          |
| Network identity | none (front by Service)      | stable DNS per pod via headless Service<br>(db-0.db.ns.svc)                |
| Storage          | shared/none                  | <b>one PVC per pod</b> , re-attached to the same ordinal on reschedule     |
| Ordering         | all at once                  | <b>ordered:</b> db-0 Ready before db-1 starts; reverse order to scale down |

```

1 apiVersion: apps/v1
2 kind: StatefulSet
3 metadata: {name: db}
4 spec:
5   serviceName: db           # the HEADLESS service giving per-pod DNS
6   replicas: 3
7   selector: {matchLabels: {app: db}}
8   template:
9     metadata: {labels: {app: db}}
10    spec:
11      containers:
12        - name: pg
13          image: postgres:16
14          volumeMounts: [{name: data, mountPath: /var/lib/postgresql/data}]
15      volumeClaimTemplates: # each pod gets ITS OWN PVC (db-data-db-0, ...-db-1)
16        - metadata: {name: data}

```

```

17     spec:
18       accessModes: ["ReadWriteOnce"]
19       resources: {requests: {storage: 50Gi}}

```

**Why ordering + stable identity matter:** clustered stateful systems do membership/replication based on identity. `db-0` is the seed/primary; `db-1` joins by contacting `db-0`. `db-...` Random names and shared storage would break leader election and replication. The PVC sticks to the ordinal so when `db-1` reschedules to a new node, it reattaches *its own* data, not someone else's.

## 9.5 DaemonSet

Runs **one pod per node** (or per node matching a selector). Use for node-level agents: log collectors (Fluent Bit), metrics (node-exporter), CNI plugins (Calico/Cilium), CSI node drivers, security agents. As nodes join/leave the cluster, the DaemonSet controller adds/removes the pod automatically.

```

1  apiVersion: apps/v1
2  kind: DaemonSet
3  metadata: {name: node-exporter}
4  spec:
5    selector: {matchLabels: {app: node-exporter}}
6    template:
7      metadata: {labels: {app: node-exporter}}
8      spec:
9        tolerations: [{operator: "Exists"}] # run even on tainted/control-plane nodes
10       containers:
11         - name: node-exporter
12           image: prom/node-exporter:v1.7.0
13           ports: [{containerPort: 9100, hostPort: 9100}]

```

## 9.6 Job & CronJob

- **Job** runs pods *to completion* (batch: migrations, ETL, one-off processing). `completions` (how many successes) + `parallelism` (how many at once) + `backoffLimit` (retries before marking failed).
- **CronJob** creates a Job on a cron schedule ("`0 2 * * *`"), with `concurrencyPolicy` (Forbid/Allow/Replace) to handle overlap and `successful/failedJobsHistoryLimit` for cleanup.

```

1  apiVersion: batch/v1
2  kind: CronJob
3  metadata: {name: nightly-report}
4  spec:
5    schedule: "0 2 * * *"
6    concurrencyPolicy: Forbid # don't start a new run if the last is still going
7    jobTemplate:
8      spec:
9        backoffLimit: 3
10       template:
11         spec:
12           restartPolicy: OnFailure
13           containers:
14             - name: report
15               image: reporter:2.1

```

16

`command: ["python", "report.py"]`

**Interview takeaway:** Deployment = stateless, interchangeable, random-named pods. StatefulSet = stable identity + ordering + per-pod persistent storage (DBs). DaemonSet = one per node (agents). Job/CronJob = run-to-completion / scheduled batch. Pick by *what the workload needs from identity, ordering, and storage*.

## 10 Kubernetes Networking

### 10.1 The Kubernetes network model (the rules)

Four mandates every CNI must satisfy:

1. **Every pod gets its own routable IP** (the "IP-per-pod" model).
2. **Pods communicate pod-to-pod without NAT** — the IP a pod sees as its own is the IP others use to reach it. One flat address space.
3. **Nodes can reach all pods** (and vice versa) without NAT.
4. Agents on a node (kubelet) can reach pods on that node.

This is deliberately simpler than Docker's per-host NAT bridge: no port-mapping gymnastics, no "which host port did this land on." The cost is you need a **CNI** plugin to make a flat network span all nodes.

### 10.2 CNI (Container Network Interface)

The kubelet calls a **CNI** plugin to wire each new pod's netns: create a veth pair, assign an IP from the node's pod CIDR, set up routes. Implementations differ in *how* they make pod IPs routable across nodes:

- **overlay (VXLAN/IP-in-IP)** — Flannel, Calico (overlay mode): encapsulate pod traffic in a tunnel between nodes. Works anywhere; small perf overhead.
- **native routing / BGP** — Calico (BGP), Cilium: program the network to route pod CIDRs directly. Faster, needs network cooperation.
- **eBPF dataplane** — Cilium: replaces iptables/kube-proxy with eBPF programs in the kernel for performance + L7 policy + observability.

### 10.3 Services --- stable virtual IPs

Pods are mortal (IPs churn). A **Service** gives a *stable* virtual IP (ClusterIP) and DNS name in front of a *dynamic* set of pods chosen by **label selector**. The Endpoints/EndpointSlice controller keeps the backing pod-IP list current; kube-proxy programs the data path.

| Type                          | What it exposes                                  | How                                                     | Use                                 |
|-------------------------------|--------------------------------------------------|---------------------------------------------------------|-------------------------------------|
| <b>ClusterIP</b><br>(default) | Internal virtual IP                              | kube-proxy DNATs VIP → random pod IP                    | Service-to-service inside cluster   |
| <b>NodePort</b>               | A static port (30000–32767) on <b>every</b> node | Node:port → ClusterIP → pod                             | Dev / on-prem without cloud LB      |
| <b>LoadBalancer</b>           | A cloud LB with an external IP                   | cloud-controller provisions an ELB/NLB → NodePort → pod | Production external access on cloud |
| <b>ExternalName</b>           | A CNAME to an external host                      | DNS only, no proxying                                   | Alias an external DB/API            |

| Type                                 | What it exposes                      | How         | Use                          |
|--------------------------------------|--------------------------------------|-------------|------------------------------|
| <b>Headless</b><br>(clusterIP: None) | No VIP; DNS returns pod IPs directly | per-pod DNS | StatefulSets, client-side LB |

```

1 apiVersion: v1
2 kind: Service
3 metadata: {name: web}
4 spec:
5   type: ClusterIP
6   selector: {app: web}      # picks pods with label app=web
7   ports:
8     - port: 80              # the Service's port
9       targetPort: 8080      # the container's port

```

## 10.4 kube-proxy: iptables vs IPVS

- › **iptables mode:** kube-proxy writes DNAT rules; a packet to the ClusterIP is rewritten by the kernel's netfilter to a randomly chosen healthy pod IP. Rules are evaluated as a **linear chain** → O(n) in number of services; fine to a few thousand services.
- › **IPVS mode:** uses kernel IPVS hash tables → **O(1)** lookup, better LB algorithms (round-robin, least-conn, consistent hash), scales to 10k+ services. Preferred for big clusters.
- › Either way, **kube-proxy is a rule programmer, not a packet forwarder** in iptables/IPVS mode — the kernel does the actual routing.

## 10.5 CoreDNS --- service discovery

Every Service gets a DNS name: `web.default.svc.cluster.local` (<service>.<namespace>.svc.cluster.local). **CoreDNS** (a cluster Deployment) answers these. Pods are configured (via `/etc/resolv.conf` search domains) so `web` resolves within the same namespace, `web.other-ns` across namespaces. Headless services return *all* pod IPs (A records); ClusterIP services return the single VIP. **DNS is a top operational risk at scale** (CoreDNS overload, `ndots:5` causing extra lookups) — see Production Concerns.

## 10.6 Ingress + Ingress controllers

A Service `type=LoadBalancer` per app is expensive (one cloud LB each) and L4 only. **Ingress** is an L7 (HTTP/HTTPS) router: host/path-based routing, TLS termination, all behind *one* load balancer. The Ingress object is just config; an **Ingress controller** (nginx-ingress, AWS ALB, Traefik, Contour) is the actual proxy pod(s) that watch Ingress objects and program themselves.

```

1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: site
5   annotations: {nginx.ingress.kubernetes.io/rewrite-target: /}
6 spec:
7   tls:
8     - hosts: [shop.example.com]
9       secretName: shop-tls      # cert+key (often from cert-manager/Let's Encrypt)
10  rules:
11    - host: shop.example.com
12      http:

```

```

13     paths:
14       - path: /api
15         pathType: Prefix
16         backend: {service: {name: api, port: {number: 80}}}
17       - path: /
18         pathType: Prefix
19         backend: {service: {name: web, port: {number: 80}}}

```

(The modern successor is the **Gateway API** — more expressive, role-oriented; mention it as “where Ingress is heading.”)

## 10.7 NetworkPolicy

By default, **all pods can talk to all pods** (flat network). **NetworkPolicy** (enforced by the CNI — Calico/Cilium; *not* all CNIs support it) is a per-pod firewall: default-deny, then allow only specific ingress/egress by pod/namespace label or CIDR. Essential for multi-tenant isolation and zero-trust.

```

1  apiVersion: networking.k8s.io/v1
2  kind: NetworkPolicy
3  metadata: {name: db-allow-api}
4  spec:
5    podSelector: {matchLabels: {app: db}}      # applies TO the db pods
6    policyTypes: [Ingress]
7    ingress:
8      - from:
9        - podSelector: {matchLabels: {app: api}} # only api pods may reach db
10         ports: [{protocol: TCP, port: 5432}]

```

## 10.8 Trace: how an external request reaches a pod

1. User → https://shop.example.com
  2. DNS → public IP of the cloud Load Balancer
  3. Cloud LB (provisioned by Service type=LoadBalancer for the ingress controller) forwards :443 to a NodePort on the worker nodes
  4. kube-proxy iptables/IPVS DNAT on that node → an Ingress-controller pod (nginx)
  5. nginx terminates TLS, matches host=shop.example.com + path=/api  
→ routes to Service "api" (ClusterIP) using the live Endpoints (pod IP list)
  6. kube-proxy DNAT: api ClusterIP → one healthy "api" pod IP (random/round-robin)
  7. CNI routes the packet to that pod (same node = local veth; remote = overlay/BGP)
  8. The pod's app container (port 8080) handles the request.
- Reverse path returns the response. Each hop is config-driven and self-updating as pods come and go (Endpoints controller keeps the IP lists current).

**Interview takeaway:** “Flat IP-per-pod, no NAT pod-to-pod; Services give stability via label selectors + kube-proxy DNAT; CoreDNS gives discovery; Ingress gives L7 host/path/TLS behind one LB; NetworkPolicy is the firewall.” Being able to trace the full external→pod path is a strong senior signal.

# 11 Config, Secrets & Storage

## 11.1 ConfigMap vs Secret

Externalize config from the image (12-factor) so the *same image* runs in dev/staging/prod with different config.

|                   | ConfigMap                                 | Secret                                                   |
|-------------------|-------------------------------------------|----------------------------------------------------------|
| For               | Non-sensitive config (URLs, flags, files) | Sensitive (passwords, tokens, TLS, registry creds)       |
| Stored in etcd as | plaintext                                 | <b>base64 (encoding, NOT encryption!)</b>                |
| Size limit        | ~1 MiB                                    | ~1 MiB                                                   |
| Types             | Opaque                                    | Opaque, kubernetes.io/tls, dockerconfigjson, SA token... |

**The #1 Secrets gotcha:** Kubernetes Secrets are only **base64-encoded by default** — `echo <val> | base64 -d` reveals them. Base64 is *encoding for binary-safety, not encryption*. To actually protect them:

1. Enable **encryption at rest** in etcd (EncryptionConfiguration with aescbc/KMS) so the API server encrypts Secret values before writing to etcd.
2. Lock down **RBAC** — anyone who can `get secrets` (or `exec` into a pod, or read etcd) sees them.
3. Prefer an **external secret manager** (Vault, AWS Secrets Manager) via the External Secrets Operator or CSI Secrets Store — secrets live outside etcd, are rotated centrally, and are injected at runtime.

## 11.2 Consuming config: env vars vs volume mounts

```

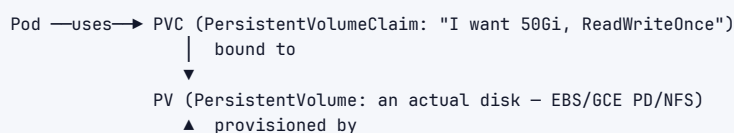
1 spec:
2   containers:
3     - name: app
4       image: myapp:1.0
5       envFrom:
6         - configMapRef: {name: app-config}    # all keys → env vars
7       env:
8         - name: DB_PASSWORD
9           valueFrom:
10            secretKeyRef: {name: db-secret, key: password} # one key → env var
11       volumeMounts:
12         - name: config-vol
13           mountPath: /etc/app                # keys → files in /etc/app/
14   volumes:
15     - name: config-vol
16       configMap: {name: app-config}

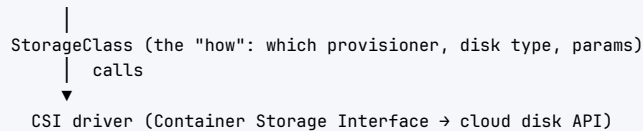
```

- ▶ **Env vars:** simple, but **captured at container start** — a ConfigMap change does *not* update a running pod's env; you must restart/roll the pod.
- ▶ **Volume mounts:** mounted as files; **auto-update** when the ConfigMap/Secret changes (with a sync delay) — though your app must re-read the file (or a sidecar reloader signals it). Use volumes for things you want to hot-reload (configs, certs); env for static startup values.

## 11.3 Volumes, PV / PVC, StorageClass, CSI

The storage abstraction separates *what an app asks for* from *how it's provisioned*:





- › **Volume** (in the pod spec) = storage mounted into a pod; `emptyDir` (ephemeral, dies with pod) vs persistent types.
- › **PV** = a cluster-level storage resource (a real disk). **PVC** = a pod's *request* for storage. K8s **binds** a PVC to a matching PV.
- › **StorageClass + dynamic provisioning**: instead of admins pre-creating PVs, a StorageClass names a **provisioner** (a CSI driver). When a PVC references it, K8s **dynamically creates** the PV/disk on demand (e.g., calls AWS to make an EBS volume). This is how StatefulSet `volumeClaimTemplates` get per-pod disks automatically.
- › **CSI (Container Storage Interface)**: the plugin standard letting any storage vendor (EBS, Ceph, Portworx, NFS) integrate without K8s core changes — analogous to CRI for runtimes and CNI for networking.

**Access modes:** `ReadWriteOnce` (RWO — one *node* mounts read-write; most block storage like EBS), `ReadOnlyMany` (ROX), `ReadWriteMany` (RWX — many nodes RW; needs NFS/CephFS/EFS). A common bug: trying to share an RWO block volume across pods on different nodes — it can't; you need RWX file storage.

**Interview takeaway:** Secrets are base64, *not* encrypted — enable etcd encryption-at-rest + tight RBAC or use an external secret manager. Storage = PVC (claim) → PV (disk) → Storage-Class (how to provision dynamically) → CSI (vendor plugin); know RWO vs RWX.

## 12 Scheduling & Resource Management

### 12.1 How the scheduler places a pod

For each unscheduled pod, kube-scheduler runs two phases over the nodes:

```

FILTER (predicates) — eliminate nodes that CANNOT run the pod:
  • Fits resources? (node allocatable ≥ pod requests for cpu/mem)
  • nodeSelector / required nodeAffinity match node labels?
  • Tolerates the node's taints?
  • Volume zone/topology compatible? PVC bindable here?
  • Node Ready & schedulable (not cordoned)? Port conflicts (hostPort)?
    ↓ (surviving "feasible" nodes)
SCORE (priorities) — rank feasible nodes 0-100, weighted:
  • LeastAllocated / MostAllocated (spread vs bin-pack)
  • BalancedResourceAllocation (even cpu:mem usage)
  • Node/Pod affinity preferences (preferredDuringScheduling...)
  • Pod anti-affinity / TopologySpread (spread replicas across zones)
  • ImageLocality (node already cached the image → faster start)
    ↓
BIND — write nodeName to the pod via API server. kubelet there starts it.
  
```

If *no* node survives filtering, the pod stays **Pending** (and Cluster Autoscaler may add a node).

### 12.2 Placement controls

- › **nodeSelector** — simplest: pod runs only on nodes with matching labels (`disktype: ssd`).
- › **Affinity / anti-affinity** — richer:
  - `nodeAffinity` (required/preferred) — attract pods to nodes by label expressions.
  - `podAffinity` — co-locate with related pods (e.g., put cache near app, same zone).
  - `podAntiAffinity` — **spread** replicas apart (don't put two replicas on the same node/zone).

→ survive a node/zone loss). The classic HA pattern.

► **Taints & tolerations** — the *repel* mechanism (inverse of affinity's attract):

- A **taint** on a node says "keep pods off unless they tolerate me." Effects: `NoSchedule` (no new pods), `PreferNoSchedule` (soft), `NoExecute` (also *evict* running pods that don't tolerate).
- A **toleration** on a pod says "I accept that taint." Use for dedicated nodes (GPU, spot instances) or to keep general pods off control-plane nodes.

```

1 # Spread replicas across zones (anti-affinity) AND tolerate GPU-tainted nodes
2 spec:
3   tolerations:
4     - {key: "nvidia.com/gpu", operator: "Exists", effect: "NoSchedule"}
5   affinity:
6     podAntiAffinity:
7       requiredDuringSchedulingIgnoredDuringExecution:
8         - labelSelector: {matchLabels: {app: web}}
9           topologyKey: topology.kubernetes.io/zone
  
```

### 12.3 Requests vs Limits, and QoS classes

This is interview gold. For each container you set:

- **request** = the amount the scheduler *reserves* (used for placement; the node must have this much free). It's the guaranteed floor.
- **limit** = the hard ceiling enforced by cgroups at runtime.

The interplay defines what happens under pressure and the pod's **QoS class**, which decides eviction order when a node runs out of memory:

| QoS class         | Condition                                                        | Eviction priority                    |
|-------------------|------------------------------------------------------------------|--------------------------------------|
| <b>Guaranteed</b> | <i>every</i> container has requests == limits for both cpu & mem | Evicted <b>last</b> (most protected) |
| <b>Burstable</b>  | at least one request set, but not all == limits                  | Evicted <b>middle</b>                |
| <b>BestEffort</b> | no requests or limits at all                                     | Evicted <b>first</b> (sacrificial)   |

Runtime behavior under the limit:

- **CPU > limit** → **throttled** (compressible) — the container is slowed, not killed. Watch for surprise p99 latency from CPU throttling even when CPU usage looks "fine" on average.
- **Memory > limit** → **OOMKilled** (incompressible) — exit 137, container restarts. Repeated → `CrashLoopBackOff`.
- **Node under memory pressure (not a single pod's limit)** → **kubelet evicts** pods in QoS order: **BestEffort** first, then **Burstable** exceeding requests, **Guaranteed** last.

**Best practice:** always set **requests** (so scheduling and QoS work and you don't get **BestEffort** surprises); set **memory limits** (prevent one pod eating the node); be **cautious with CPU limits** (throttling causes latency — many teams set CPU *requests* but omit CPU *limits*). For critical workloads, set `requests==limits` → **Guaranteed**.

### 12.4 Priority & preemption

A **PriorityClass** assigns a numeric priority to pods. When a high-priority pod can't schedule (no room), the scheduler may **preempt** (evict) lower-priority pods to make space. Used so critical control-plane/system pods always get capacity over batch jobs.

**Interview takeaway:** requests = scheduler reservation/floor; limits = cgroup ceiling. Guaranteed (req=lim) > Burstable > BestEffort for survival. CPU over limit throttles, memory over limit OOM-kills. This one paragraph separates people who've run K8s from people who've read about it.

## 13 Autoscaling

Three independent autoscalers, often confused — know exactly what each scales:

|     |                                   |                                     |
|-----|-----------------------------------|-------------------------------------|
| HPA | scales the NUMBER OF POD REPLICAS | (more copies of your app)           |
| VPA | scales the SIZE OF EACH POD       | (bigger requests/limits per pod)    |
| CA  | scales the NUMBER OF NODES        | (more machines for pods to land on) |

### 13.1 Horizontal Pod Autoscaler (HPA)

Watches a metric (default: CPU/memory via metrics-server; or custom/external via Prometheus Adapter/KEDA) and adjusts a Deployment/StatefulSet's replicas.

The control formula:

```
1 desiredReplicas = ceil( currentReplicas × ( currentMetric / targetMetric ) )
2
3 e.g., target CPU 50%, current avg 80% across 3 pods:
4     ceil(3 × 80/50) = ceil(4.8) = 5 replicas
```

- ▶ Reads metrics every ~15s; **scale-up is responsive, scale-down is conservative** (default 5-min stabilization window) to avoid flapping.
- ▶ **KEDA** extends HPA to event-driven sources: scale on Kafka lag, SQS queue depth, Prometheus query, cron — and crucially can **scale to zero**.

```
1 apiVersion: autoscaling/v2
2 kind: HorizontalPodAutoscaler
3 metadata: {name: web}
4 spec:
5   scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: web}
6   minReplicas: 3
7   maxReplicas: 20
8   metrics:
9     - type: Resource
10     resource: {name: cpu, target: {type: Utilization, averageUtilization: 60}}
```

### 13.2 Vertical Pod Autoscaler (VPA)

Right-sizes **requests/limits** per pod based on observed usage (recommends, and in Auto mode applies — historically requiring a pod restart since you can't change a running pod's resources; in-place resize is landing in newer K8s). Use VPA to *find correct requests*; it **conflicts with HPA on the same CPU/memory metric** (both fighting over the same signal) — so pair HPA (on CPU) with VPA in recommendation-only mode, or VPA on memory + HPA on a custom metric.

### 13.3 Cluster Autoscaler (CA)

Watches for **Pending pods that don't fit** any node → calls the cloud API to add a node to a node group → the new node registers → scheduler places the pending pod. Scale-down: a node underutilized (< ~50% requested) for ~10 min, whose pods can move elsewhere, is drained and terminated. **Karpenter** (AWS) is the modern alternative — provisions right-sized nodes per pending pod's needs instead of fixed node groups, faster and more bin-packing-efficient.

**Interview takeaway:** HPA scales pods (out), VPA sizes pods (up), CA scales nodes. The production combo is **HPA + Cluster Autoscaler/Karpenter**: HPA adds pods → some go Pending → CA adds nodes. Don't run HPA and VPA on the same resource metric.

## 14 Rollouts, Probes & Self-Healing

### 14.1 The three probes (and the exact failure each prevents)

The kubelet runs three independent health checks per container. Each has a distinct job and a distinct failure mode it guards against. Each supports httpGet, tcpSocket, exec, or grpc, with initialDelaySeconds, periodSeconds, failureThreshold, etc.

| Probe            | Question                          | On failure                                            | Failure it prevents                                                   |
|------------------|-----------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------|
| <b>startup</b>   | "Has it finished booting yet?"    | keep waiting; if it never passes → kill               | <b>Slow-starting apps</b> killed prematurely by liveness              |
| <b>liveness</b>  | "Is it alive or wedged?"          | <b>restart the container</b>                          | A <b>deadlocked</b> process that's running but stuck forever          |
| <b>readiness</b> | "Can it serve traffic right now?" | <b>remove pod from Service Endpoints</b> (no restart) | Sending traffic to a pod that's <b>warming up or temporarily busy</b> |

```

1 startupProbe:                                # gives a slow app up to 5min to boot
2   httpGet: {path: /healthz, port: 8080}
3   failureThreshold: 30
4   periodSeconds: 10
5 livenessProbe:                                # restart if wedged
6   httpGet: {path: /healthz, port: 8080}
7   periodSeconds: 10
8   failureThreshold: 3
9 readinessProbe:                               # pull from LB if not ready (e.g., DB down)
10  httpGet: {path: /ready, port: 8080}
11  periodSeconds: 5
12  failureThreshold: 3

```

The crucial distinctions, with scenarios:

- **Readiness ≠ Liveness.** Readiness failing → the pod is *temporarily* removed from the Service's Endpoints, so it gets **no traffic but is NOT restarted** (it can recover — e.g., a transient downstream dependency). Liveness failing → the container is **restarted** (it's presumed unrecoverable, e.g., deadlocked).
- **The classic outage from getting them backwards:** if you make livenessProbe depend on a downstream DB, then when the DB blips, *every* pod's liveness fails → K8s restarts *all* of them simultaneously → a thundering-herd restart storm and a far worse outage. **Rule: liveness should check only the process itself (am I wedged?); readiness checks dependencies (can I serve?).**
- **Startup probe** exists so a 2-minute JVM/ML-model boot isn't repeatedly killed by an aggressive liveness probe; liveness/readiness don't start until startup passes.

## 14.2 Rolling update / rollback (recap with commands)

```

1 kubectl set image deploy/web web=myapp:1.6 # triggers a rolling update
2 kubectl rollout status deploy/web           # watch it progress (blocks until done/failed)
3 kubectl rollout history deploy/web          # list revisions
4 kubectl rollout undo deploy/web             # roll back to previous revision
5 kubectl rollout undo deploy/web --to-revision=3

```

Mechanics are in the [Deployment](#) section: new ReplicaSet scaled up while old scales down, gated by readiness and maxSurge/maxUnavailable; rollback re-scales the retained old ReplicaSet.

## 14.3 Deployment strategies

| Strategy                       | How                                            | Rollback                   | Cost         | Downtime             | Notes                                                                           |
|--------------------------------|------------------------------------------------|----------------------------|--------------|----------------------|---------------------------------------------------------------------------------|
| <b>RollingUpdate</b> (default) | Gradually replace old pods with new            | Re-roll (minutes)          | None extra   | Zero (if configured) | K8s built-in; mixed versions live simultaneously                                |
| <b>Recreate</b>                | Kill all old, then start all new               | Re-deploy old              | Minimal      | <b>Yes</b>           | When two versions can't coexist (schema lock)                                   |
| <b>Blue-Green</b>              | Run v2 alongside v1; flip the Service selector | <b>Instant</b> (flip back) | 2× capacity  | Zero                 | Test v2 fully before flip; instant rollback                                     |
| <b>Canary</b>                  | Send 1–5% traffic to v2, watch metrics, ramp   | Fast (drop %)              | ~small extra | Zero                 | Validates on real traffic; needs traffic-splitting (mesh/Ingress/Argo Rollouts) |

K8s natively does Rolling and Recreate; **blue-green** and **canary** need extra tooling (two Deployments + Service flip, a service mesh for weighted routing, or **Argo Rollouts**/Flagger for automated canary analysis). FAANG production leans **canary with automated analysis** (Netflix Kayenta) — auto-rollback if error rate/latency degrades.

## 14.4 Self-healing --- what actually happens

- ▶ **Container crashes** → kubelet restarts it per restartPolicy (Always for Deployments), with **exponential backoff** (10s, 20s, 40s... capped at 5min). Repeated crashes within backoff = **CrashLoopBackOff** status.
- ▶ **Pod becomes unhealthy (liveness)** → restart the container.
- ▶ **Node dies / NotReady** → after a grace period, Node controller marks its pods for deletion; the ReplicaSet notices the shortfall and creates replacements; scheduler places them on healthy nodes.
- ▶ **Wrong replica count** (manual scale-down of a pod) → ReplicaSet immediately recreates to match desired.

All of this is the **reconciliation loop** — not special-case recovery code. That's the recurring theme to voice.

**Interview takeaway:** liveness = restart if wedged (check *self*); readiness = pull from LB if can't serve (check *dependencies*); startup = grace for slow boots. The trap is putting dependency checks in liveness → cluster-wide restart storms. Self-healing is reconciliation, not magic.

## 15 RBAC & Security

### 15.1 RBAC (Role-Based Access Control)

Four objects, two pairs (rules vs binding, namespace vs cluster-wide):

```

1 ROLE           = a set of allowed verbs on resources, WITHIN a namespace
2 CLUSTERROLE    = same, but cluster-wide (or for cluster-scoped resources: nodes, PVs)
3 ROLEBINDING     = grants a Role/ClusterRole to a subject IN a namespace
4 CLUSTERROLEBINDING = grants a ClusterRole to a subject CLUSTER-WIDE
5 SUBJECTS = User | Group | ServiceAccount

```

```

1 # Role: read-only pods/logs in namespace "team-a"
2 apiVersion: rbac.authorization.k8s.io/v1
3 kind: Role
4 metadata: {namespace: team-a, name: pod-reader}
5 rules:
6 - apiGroups: [""]
7   resources: ["pods", "pods/log"]
8   verbs: ["get", "list", "watch"]
9
10 apiVersion: rbac.authorization.k8s.io/v1
11 kind: RoleBinding
12 metadata: {namespace: team-a, name: read-pods}
13 subjects:
14 - kind: ServiceAccount
15   name: ci-bot
16   namespace: team-a
17 roleRef: {kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io}

```

- › **RBAC is purely additive and default-deny** — no DENY rules; you grant least privilege. A request is allowed if *any* binding permits it.
- › **ServiceAccount** = the identity a *pod* uses to call the API server (its token is auto-mounted, unless disabled). Give each workload its own SA with minimal rights — never the default SA with broad access. A compromised pod with cluster-admin via its SA is game over.
- › Mistake to avoid: binding cluster-admin to a workload's SA "to make it work."

### 15.2 Pod / workload security

- › **securityContext** (per pod/container): `runAsNonRoot`, `runAsUser`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, drop capabilities, seccomp profile `RuntimeDefault`. (See [Container Internals](#).)
- › **Pod Security Admission (PSA)** — built-in admission controller enforcing three levels per namespace via labels: **privileged** (anything), **baseline** (block known-bad), **restricted** (hardened: non-root, no privilege escalation, drop caps). Replaced the removed `PodSecurityPolicy`. For richer policy, use **OPA Gatekeeper** or **Kyverno** (write custom admission rules: "no :latest tags," "all images from our registry," "require resource limits").
- › **NetworkPolicy** for network-level isolation (above).

### 15.3 Secrets at rest

Recap the gotcha: Secrets are base64 in etcd by default. For production: **encryption at rest** (EncryptionConfiguration, ideally KMS-backed so the data-encryption key is itself protected), tight RBAC on secrets, and prefer external managers (Vault/AWS SM via External Secrets Operator). Audit who can get secrets.

**Interview takeaway:** RBAC is additive/default-deny — grant least privilege; each pod gets its own minimal-rights ServiceAccount. Harden pods via securityContext (non-root, drop caps, read-only FS) and enforce it with Pod Security Admission / Kyverno. Encrypt Secrets at rest.

## 16 Operators & CRDs

### 16.1 CRDs --- extending the API

A **CustomResourceDefinition** teaches the API server a *new resource kind*. After applying a CRD for kind: Database, you can `kubectl get databases` and `kubectl apply` a Database object exactly like a built-in. The API server now stores and serves your custom type (validation, RBAC, watch all come free).

```

1  apiVersion: apiextensions.k8s.io/v1
2  kind: CustomResourceDefinition
3  metadata: {name: databases.example.com}
4  spec:
5    group: example.com
6    scope: Namespaced
7    names: {kind: Database, plural: databases, singular: database, shortNames: [db]}
8    versions:
9      - name: v1
10        served: true
11        storage: true
12        schema:
13          openAPIV3Schema:
14            type: object
15            properties:
16              spec:
17                type: object
18                properties:
19                  engine: {type: string, enum: [postgres, mysql]}
20                  version: {type: string}
21                  storageGiB: {type: integer}

```

A CRD by itself only *stores* the object — nothing happens. You need a controller.

### 16.2 The Operator pattern

An **Operator** = a **CRD + a custom controller** that encodes the *operational knowledge* of running a specific application as code. It runs the same reconciliation loop the built-in controllers do, but for your domain:

```

You apply:  kind: Database, spec: {engine: postgres, version: 16, storageGiB: 100}
Operator's controller loop:
  observe desired Database vs actual world →
  create a StatefulSet + Service + PVC + Secret,
  run init/bootstrap, configure replication,
  take scheduled backups, handle failover & version upgrades,
  update Database.status with health.

```

So instead of a human DBA performing backups/failover/upgrades, the operator does it continuously. Real examples: **prometheus-operator** (kind: Prometheus), **cert-manager** (kind: Certificate → auto-provisions TLS from Let's Encrypt and renews), **etcd/PostgreSQL/Kafka operators**, cloud operators (ACK/Crossplane provisioning cloud infra via CRDs).

Build operators with the **Operator SDK / Kubebuilder** (Go) or **Metacontroller/KOPF** (lighter). The mental model: *Kubernetes is itself a platform of controllers; an operator is just one more controller you write to manage a Custom Resource the same level-triggered way.*

**Interview takeaway:** CRD = a new API type; Operator = CRD + a controller that automates day-2 ops (backup, failover, upgrade) for a stateful app. It's the same reconciliation pattern as core K8s, extended to your domain — "encode your runbook as a controller."

## 17 Helm

This is the deepest section because Helm is the biggest gap in the moderate-level material. **Helm is the package manager for Kubernetes** — "apt/yum/brew for K8s." It solves two distinct problems: (1) **templating** (parameterize manifests so one package serves dev/staging/prod and arbitrary users), and (2) **release management** (track an install as a versioned, atomically upgrade-able, rollback-able unit).

### 17.1 The problem Helm solves: YAML sprawl

A real app is a Deployment + Service + Ingress + ConfigMap + Secret + HPA + ServiceAccount + PodDisruptionBudget + NetworkPolicy — easily 8–12 manifests, ~1500–2500 lines. Now you need that per environment (dev/staging/prod differ in replicas, image tag, resources, host-names, feature flags). Hand-copying YAML per env → drift, errors, and no atomic "deploy this whole app as a unit / roll the whole thing back." Helm packages all of it into one parameterized **chart** and manages installs as named **releases**.

### 17.2 Chart structure

```
mychart/
├── Chart.yaml           # chart metadata: name, version, appVersion, dependencies
├── values.yaml          # DEFAULT configuration values (the public "API" of the chart)
├── values.schema.json   # (optional) JSON Schema to validate user-supplied values
├── templates/           # the templated K8s manifests
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── configmap.yaml
│   ├── _helpers.tpl     # reusable named template snippets (no leading "_" → not rendered alone)
│   ├── NOTES.txt        # post-install message printed to the user
│   └── tests/            # `helm test` hooks
├── charts/              # subcharts (vendored dependencies) live here
└── .helmignore          # like .gitignore for `helm package`
```

- › **Chart.yaml** — identity + versions. version is the *chart* version (SemVer); appVersion is the *app* version it deploys; dependencies lists subcharts.
- › **values.yaml** — the chart's tunable knobs and their defaults; this *is* the chart's interface.
- › **templates/** — Go-templated manifests rendered against values + built-in objects.
- › **\_helpers.tpl** — files starting with \_ aren't rendered as manifests; they hold defined named templates reused across the chart.
- › **NOTES.txt** — rendered and printed after install (e.g., "your app is at http://...").

```

1 # Chart.yaml
2 apiVersion: v2
3 name: mychart
4 description: A web app
5 type: application
6 version: 1.2.0      # chart version (bump on chart changes)
7 appVersion: "2.5.1" # the app image version this chart deploys
8 dependencies:
9   - name: postgresql
10     version: "13.x.x"
11     repository: "https://charts.bitnami.com/bitnami"
12     condition: postgresql.enabled

```

### 17.3 Templating (Go templates)

Templates are Go text/template with the **Sprig** function library plus Helm-specific objects. Values and built-ins are injected:

| Reference                                | Source                                           |
|------------------------------------------|--------------------------------------------------|
| {{ .Values.x }}                          | from values.yaml / -f / --set (merged)           |
| {{ .Release.Name }}                      | the release name (helm install myrel ...)        |
| {{ .Release.Namespace }}                 | target namespace                                 |
| {{ .Chart.Name }} / {{ .Chart.Version }} | from Chart.yaml                                  |
| {{ .Capabilities.KubeVersion }}          | cluster capabilities (e.g., gate by API version) |
| {{ .Files.Get "config.json" }}           | read a non-template file in the chart            |

#### Pipelines & functions (data flows left→right through |):

- ▶ {{ .Values.name | default "web" | quote }} — supply a default, then wrap in quotes.
- ▶ {{ .Values.replicas | int }} — type coercion.
- ▶ nindent N / indent N — indent a block to fit YAML (crucial for embedding multi-line blocks).
- ▶ {{ toYaml .Values.resources | nindent 12 }} — serialize a values map to YAML at the right indent (the single most-used idiom).
- ▶ {{ include "mychart.labels" . | nindent 4 }} — render a named template and indent it.
- ▶ whitespace control: {{- / -}} trim preceding/following whitespace/newlines.

**Named templates** (define in \_helpers.tpl, include to use — prefer include over template because include's output can be piped into nindent):

```

1 {{/* templates/_helpers.tpl */}}
2 {{- define "mychart.fullname" -}}
3 {{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" -}}
4 {{- end -}}
5
6 {{- define "mychart.labels" -}}
7 app.kubernetes.io/name: {{ .Chart.Name }}
8 app.kubernetes.io/instance: {{ .Release.Name }}
9 app.kubernetes.io/managed-by: {{ .Release.Service }}
10 helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
11 {{- end -}}

```

#### Conditionals & loops:

```

1 {{- if .Values.ingress.enabled }}
2 apiVersion: networking.k8s.io/v1
3 kind: Ingress
4 # ...
5 {{- else }}
6 # nothing rendered
7 {{- end }}
8
9 env:
10 {{- range $key, $val := .Values.env }}      # loop over a map
11   - name: {{ $key }}
12     value: {{ $val | quote }}
13 {{- end }}
14 {{- range .Values.extraHosts }}            # loop over a list ($ = root scope)
15   - host: {{ . }}
16 {{- end }}

```

## 17.4 Full worked mini-chart (web app: Deployment + Service)

values.yaml:

```

1 replicaCount: 2
2 image:
3   repository: myorg/web
4   tag: "1.5.0"
5   pullPolicy: IfNotPresent
6 service:
7   type: ClusterIP
8   port: 80
9   targetPort: 8080
10 resources:
11   requests: {cpu: "100m", memory: "128Mi"}
12   limits:   {cpu: "500m", memory: "256Mi"}
13 ingress:
14   enabled: false
15 env:
16   LOG_LEVEL: info

```

templates/deployment.yaml:

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: {{ include "mychart.fullname" . }}
5   labels:
6     {{- include "mychart.labels" . | nindent 4 }}
7 spec:
8   replicas: {{ .Values.replicaCount }}
9   selector:
10     matchLabels:
11       app.kubernetes.io/name: {{ .Chart.Name }}
12       app.kubernetes.io/instance: {{ .Release.Name }}
13   template:
14     metadata:
15       labels:

```

```

16     {{- include "mychart.labels" . | nindent 8 }}
17   spec:
18     containers:
19     - name: {{ .Chart.Name }}
20       image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
21       imagePullPolicy: {{ .Values.image.pullPolicy }}
22       ports:
23       - containerPort: {{ .Values.service.targetPort }}
24       env:
25         {{- range $k, $v := .Values.env }}
26         - name: {{ $k }}
27           value: {{ $v | quote }}
28         {{- end }}
29     resources:
30       {{- toYaml .Values.resources | nindent 12 }}

```

templates/service.yaml:

```

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: {{ include "mychart.fullname" . }}
5    labels:
6      {{- include "mychart.labels" . | nindent 4 }}
7  spec:
8    type: {{ .Values.service.type }}
9    selector:
10     app.kubernetes.io/name: {{ .Chart.Name }}
11     app.kubernetes.io/instance: {{ .Release.Name }}
12    ports:
13     - port: {{ .Values.service.port }}
14       targetPort: {{ .Values.service.targetPort }}

```

Install for prod with overrides:

```

1  helm install web ./mychart -n prod \
2    --set replicaCount=6 \
3    --set image.tag=1.5.1 \
4    -f values-prod.yaml

```

## 17.5 Releases & revisions

A **release** is a named install of a chart into a namespace. Helm 3 stores release state as a **Secret in that namespace** (one per revision, named `sh.helm.release.v1.<release>.v<n>`). Each upgrade creates a **new revision**, enabling rollback.

```

1  helm install web ./mychart           # creates release "web", revision 1
2  helm upgrade web ./mychart           # revision 2
3  helm upgrade web ./mychart --set x=y # revision 3
4  helm history web                     # list all revisions with status
5  helm rollback web 1                  # roll back to revision 1 (creates revision 4!)
6  helm uninstall web                   # delete the release (and its resources)

```

- › **--atomic**: if the upgrade fails (e.g., a pod never becomes ready within `--timeout`), Helm automatically rolls back to the prior revision — no half-applied state.
- › **--dry-run / helm template**: render manifests *without* applying — review the exact YAML. `--dry-run` consults the cluster; `helm template` is purely client-side.
- › **--wait**: block until resources are ready before reporting success.

**Lifecycle hooks** run jobs at points in the release lifecycle via annotations: `pre-install`, `post-install`, `pre-upgrade`, `post-upgrade`, `pre-delete`, `post-delete`. Classic use: a `pre-upgrade` Job that runs DB migrations before the new pods roll, or a `pre-delete` Job to back up data.

```

1 metadata:
2   annotations:
3     "helm.sh/hook": pre-upgrade
4     "helm.sh/hook-weight": "0"
5     "helm.sh/hook-delete-policy": hook-succeeded

```

## 17.6 Values precedence

When the same key is set in multiple places, the **last/most-specific wins**:

```

chart's values.yaml (lowest)
  → parent chart values overriding a subchart
  → -f myvalues.yaml (each subsequent -f overrides the prior)
  → --set key=val (highest — beats files)

```

So `--set` beats `-f` beats `values.yaml`. Per-environment pattern: keep `values-dev.yaml`, `values-staging.yaml`, `values-prod.yaml` and pick one with `-f`. **Subchart overriding**: a parent overrides a subchart's value by nesting under the subchart's name:

```

1 # parent values.yaml
2 postgresql:      # <- subchart name
3   auth:
4     database: appdb # overrides the postgresql subchart's .Values.auth.database

```

Global values (`global:`) are visible to all subcharts.

## 17.7 Dependencies (subcharts)

Declare dependencies in `Chart.yaml`, then vendor them into charts/:

```

1 helm repo add bitnami https://charts.bitnami.com/bitnami
2 helm dependency update ./mychart # reads Chart.yaml deps → downloads into charts/
3 helm dependency build  ./mychart # rebuild from Chart.lock

```

`condition: postgresql.enabled` lets users toggle a subchart on/off. This is how you compose an app that bundles its database, redis, etc., yet still let users swap to an external one.

## 17.8 Essential commands

| Command                                                                    | Purpose                               |
|----------------------------------------------------------------------------|---------------------------------------|
| <code>helm install &lt;rel&gt; &lt;chart&gt; [-f vals] [--set k=v]</code>  | Create a release                      |
| <code>helm upgrade &lt;rel&gt; &lt;chart&gt; [--install] [--atomic]</code> | Upgrade (or install if absent)        |
| <code>helm rollback &lt;rel&gt; &lt;rev&gt;</code>                         | Revert to a prior revision            |
| <code>helm uninstall &lt;rel&gt;</code>                                    | Delete a release                      |
| <code>helm list [-A]</code>                                                | List releases (all namespaces)        |
| <code>helm template &lt;chart&gt;</code>                                   | Render manifests locally (no cluster) |
| <code>helm lint &lt;chart&gt;</code>                                       | Static-check the chart                |
| <code>helm get manifest/values/notes &lt;rel&gt;</code>                    | Inspect what was deployed             |
| <code>helm history &lt;rel&gt;</code>                                      | Show revision history                 |
| <code>helm dependency update &lt;chart&gt;</code>                          | Resolve subcharts                     |
| <code>helm repo add/update/search</code>                                   | Manage chart repositories             |

## 17.9 Helm 2 vs Helm 3 (and why Tiller mattered)

|                  | Helm 2                                                            | Helm 3                                                  |
|------------------|-------------------------------------------------------------------|---------------------------------------------------------|
| Server component | <b>Tiller</b> (in-cluster, ran your manifests)                    | <b>None</b> — pure client; talks to API server directly |
| Security         | Tiller often had cluster-admin → a huge privilege-escalation hole | Uses <i>your</i> kubeconfig/RBAC — least privilege      |
| Release storage  | ConfigMaps in kube-system                                         | Secrets in the release's namespace                      |
| Release scope    | Global names                                                      | <b>Namespace-scoped</b> names                           |
| CRDs             | Hooky                                                             | First-class crds/ directory                             |

**Why removing Tiller mattered:** Tiller was a privileged in-cluster daemon that applied charts on your behalf — anyone who could reach Tiller could deploy anything as cluster-admin, bypassing RBAC. Helm 3 made Helm a thin client that uses *your* credentials, so the security story finally matched normal RBAC. Always “Helm 3” today.

## 17.10 Helm vs Kustomize

Two philosophies for “manage K8s YAML across environments”:

|                    | Helm                                            | Kustomize                                                 |
|--------------------|-------------------------------------------------|-----------------------------------------------------------|
| Mechanism          | <b>Templating</b> (Go templates + values)       | <b>Overlays</b> (patch a base with env-specific diffs)    |
| Mental model       | “Fill in the blanks”                            | “Start from a base, layer patches”                        |
| Packaging/sharing  | Charts + repos (great for redistributable apps) | No packaging; just directories                            |
| Release mgmt       | Yes — versioned releases, rollback, hooks       | No — it just emits YAML ( <code>kubectl apply -k</code> ) |
| Values             | One values file, many overrides                 | <code>kustomization.yaml</code> overlays per env          |
| Complexity         | Can get “template soup” with heavy logic        | Pure YAML; no templating language to learn                |
| Built into kubectl | No (separate binary)                            | <b>Yes</b> ( <code>kubectl apply -k</code> )              |

```

1 # Kustomize: overlays/prod/kustomization.yaml
2 resources: [../..base]
3 patchesStrategicMerge: [replica-count.yaml] # patch the base Deployment's replicas
4 images:
5   - name: myorg/web
6     newTag: 1.5.1

```

**When to use which:** **Helm** when you need *packaging/distribution* (sharing a reusable app like Prometheus or your DB operator), *parameterization for many users*, and *release lifecycle* (rollback/hooks). **Kustomize** when you own all the YAML, want *no templating language*, and just need clean per-environment variation. Common hybrid: **Helm to render, Kustomize to post-patch** (`helm template | kubectl apply -k`), or use Helm for third-party charts + Kustomize for your own services. ArgoCD/Flux support both.

## 17.11 Best practices

- › Pin chart + subchart versions; commit `Chart.lock`.
- › Keep templates dumb; push logic into values + `_helpers.tpl`. Avoid "template soup."
- › Always `helm lint` + `helm template | kubeval/kubeconform` in CI; review `--dry-run` diffs (`helm diff` plugin) before upgrades.
- › Use `--atomic --timeout` in pipelines so failed upgrades auto-rollback.
- › Add `values.schema.json` to validate user input and fail fast.
- › Set sensible resources, probes, and `PodDisruptionBudget` defaults in the chart.

**Interview takeaway:** Helm = templating + release management (versioned, rollbackable installs). Helm 3 killed Tiller → uses your RBAC. Helm vs Kustomize: templating-with-packaging-and-releases vs overlay-patching-built-into-kubectl. Pick Helm for redistributable/parameterized apps with lifecycle needs; Kustomize for your own YAML with simple env variation.

## 18 kubectl & Debugging

### 18.1 kubectl essentials

```

1 kubectl get pods -o wide -n prod # list pods + node/IP
2 kubectl get pods -w # watch changes live
3 kubectl describe pod web-abc # events + state + probe results (start HERE)
4 kubectl logs web-abc -c app --previous # logs of the PREVIOUS (crashed) container
5 kubectl logs -f deploy/web # stream a deployment's logs
6 kubectl exec -it web-abc -- sh # shell into a container
7 kubectl get events --sort-by=.lastTimestamp # cluster-wide recent events
8 kubectl apply -f manifest.yaml # declarative create/update
9 kubectl rollout status/undo deploy/web # rollout control
10 kubectl top pod/node # live CPU/mem (needs metrics-server)
11 kubectl port-forward svc/web 8080:80 # tunnel a service to localhost
12 kubectl debug -it web-abc --image=busybox --target=app # ephemeral debug container
13 kubectl explain pod.spec.containers # field docs from the API schema

```

### 18.2 Debugging a broken pod --- the systematic flow

The universal first three commands: **describe** → **logs** → **events**. The pod's status word tells you which branch:

```

1. kubectl get pods           → note the STATUS column
2. kubectl describe pod <p>   → read the Events at the bottom + container State/Reason
3. kubectl logs <p> [-c c] [--previous] → app's own error output
4. kubectl get events --sort-by=.lastTimestamp

```

**CrashLoopBackOff** (container starts then exits repeatedly; kubelet backs off):

- › It's not an error type — it's "your container keeps dying." Find *why it exits*.
- › `kubectl logs <p> --previous` → the crashed container's last output (the actual stack trace / panic). This is the key command.
- › `describe` → exit code: **137** = OOMKilled (raise memory limit / fix leak), **1/2** = app error (bad config, missing env/secret, failed DB connect), **126/127** = command not found/not executable (bad ENTRYPOINT).
- › Common causes: missing ConfigMap/Secret, bad env, failed migration, liveness probe too aggressive (killing a slow-booting app → add a **startup probe**), wrong command.
- › If the container won't stay up to debug: `kubectl debug` an ephemeral container, or command: `["sleep", "3600"]` override to get a shell.

**ImagePullBackOff / ErrImagePull** (can't fetch the image):

- › `describe` shows the pull error. Causes: typo'd image/tag, **tag doesn't exist** in the registry, **private registry without an imagePullSecret**, registry auth expired, rate-limited (Docker Hub), wrong registry host, network/egress blocked.
- › Fix: verify image: exactly, `kubectl create secret docker-registry ...` and reference it in the pod's `imagePullSecrets`, check the node can reach the registry.

**Pending** (scheduler can't place it — `describe` shows "FailedScheduling" with the reason):

- › **Insufficient cpu/memory** → no node has room for the pod's *requests*. Lower requests, add nodes, or rely on Cluster Autoscaler.
- › **No node matches** nodeSelector/affinity, or **untolerated taint** on the only fitting nodes.
- › **PVC unbound** → no PV/StorageClass can satisfy the claim (wrong access mode, zone mismatch).
- › **All nodes cordoned** or hostPort conflict.
- › Read the exact `describe` reason — it tells you which predicate failed.

**Other quick reads:** ContainerCreating stuck → image still pulling or volume mount failing (`describe`); Terminating stuck → finalizer or long `terminationGracePeriodSeconds`/stuck `preStop`; 0/1 Ready but Running → readiness probe failing (check the probe + app /ready).

**Interview takeaway:** The debugging answer interviewers want: "describe for events/state, logs --previous for the crash output, get events for context — then branch on the status word: CrashLoop = why does it exit (check exit code: 137=OOM, logs for app errors); ImagePullBackOff = name/tag/secret/registry; Pending = scheduling (requests/taints/affinity/PVC)."

## 19 Production Concerns & What Breaks at Scale

What's smooth in a 3-node demo and what bites at 1000s of nodes:

### 19.1 etcd limits

- › etcd holds *all* cluster state and is a Raft-quorum CP store; it has a **practical DB size limit (~8 GB)** and degrades with high write churn. Too many objects (huge numbers of Secrets/-ConfigMaps/Events, CRD spam, runaway controllers) bloats etcd and slows the whole API server.

- › Mitigation: limit Events retention, compact/defrag etcd, watch object counts, isolate etcd on fast disks (it's fsync-latency sensitive), back it up relentlessly.

## 19.2 Too many pods / node density

- › The kubelet has a **default ~110 pods/node** cap; the cluster has practical ceilings (historically ~5,000 nodes / ~150k pods per cluster). Beyond that → API server and etcd pressure.
- › Mitigation: **multi-cluster** (split by team/region/blast-radius) rather than one giant cluster; right-size pods; avoid tiny pods that waste the per-pod overhead.

## 19.3 DNS at scale

- › CoreDNS becomes a hotspot: every service call may trigger DNS, and the default `ndots:5` in pods causes **multiple failed lookups** before the real one (each name tried with several search domains). Under load this overwhelms CoreDNS → cluster-wide latency/timeouts that *look* like app bugs.
- › Mitigation: **NodeLocal DNSCache** (per-node DNS cache), tune `ndots`, autoscale CoreDNS, use FQDNs to skip search-domain expansion.

## 19.4 Resource starvation & noisy neighbors

- › Pods without requests get **BestEffort** QoS → first evicted, and can starve neighbors. One pod without a CPU limit can throttle the node; without a memory limit can OOM the node.
- › Mitigation: enforce requests/limits via **LimitRange + ResourceQuota** per namespace; PodDisruptionBudgets so voluntary disruptions (drains, upgrades) don't take out too many replicas at once.

## 19.5 Control-plane & rollout pitfalls

- › A bad readiness probe or image stalls a rollout (good — it won't proceed); but a bad *liveness* probe tied to a dependency causes **fleet-wide restart storms**.
- › Thundering-herd image pulls on a big scale-up saturate the registry → use a pull-through cache / pre-pulled images.
- › Single-zone clusters die with the zone → spread across **availability zones** (topology spread + multi-AZ node groups).

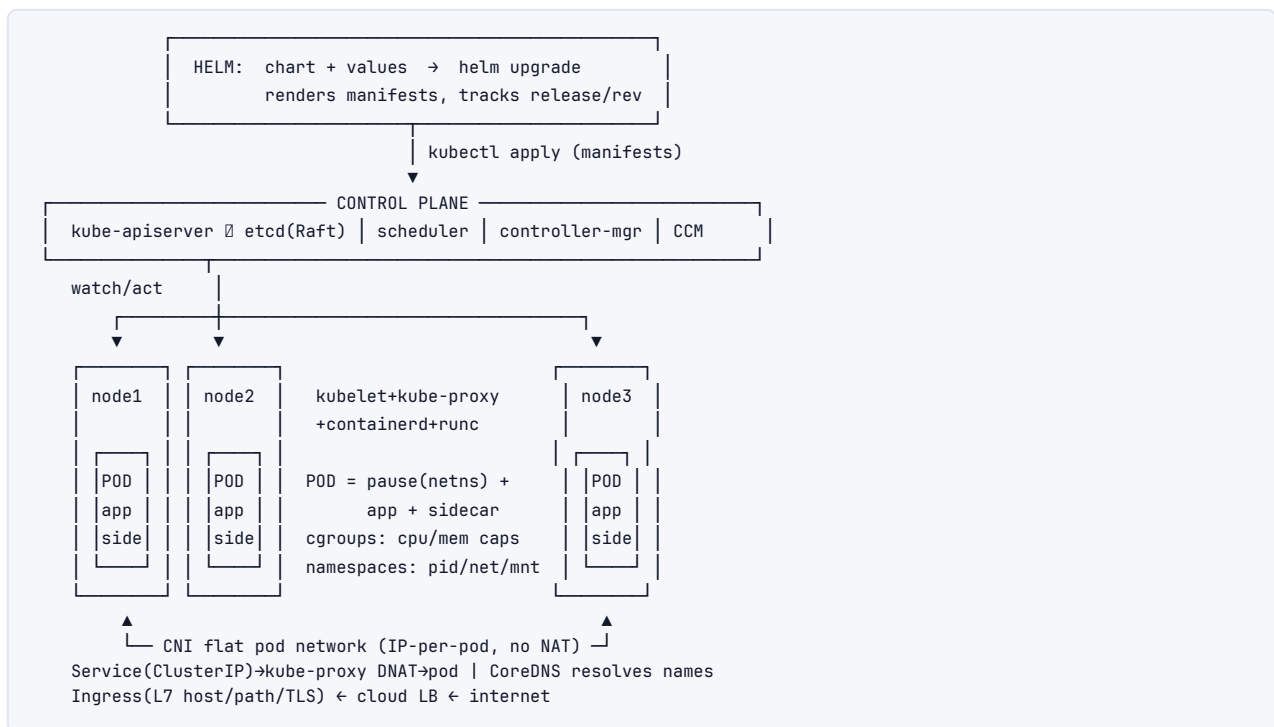
## 19.6 Multi-cluster / multi-region

- › Reasons: blast-radius isolation, regional latency/compliance, scaling past per-cluster limits, blue-green at the cluster level.
- › Tools: **Cluster API** (declarative cluster lifecycle), **fleet managers** (Rancher, Anthos, EKS/GKE fleets), GitOps (ArgoCD ApplicationSets) to fan one config across clusters, global load balancing + service mesh (Istio multi-cluster) for cross-cluster traffic. Trade-off: operational complexity and cross-cluster networking/identity.

**Interview takeaway:** At scale the failure modes are etcd size/latency, pod-density ceilings, **DNS (CoreDNS + ndots)**, and resource starvation from missing requests/limits. The standard answers: multi-cluster for blast radius, NodeLocal DNSCache, enforce quotas/limits, spread across AZs, and treat etcd backups as sacred.

# 20 Architecture / Diagrams

## 20.1 The full stack in one picture



## 20.2 Inside a container (the kernel view)

a "container" is ONE process group on the host:

```
PID-ns:  app is PID 1, can't see host PIDs
NET-ns:  own eth0/IP (from pause container)
MNT-ns:  own / = image rootfs (OverlayFS merged)
UTS/IPC/USER-ns: own hostname / IPC / UID mapping
— cgroup: cpu.max, memory.max (throttle / OOMKill) —
— capabilities dropped, seccomp/AppArmor applied —
```

## 20.3 Rolling update (ReplicaSet choreography)

```
Deployment "web" (replicas=4, maxSurge=1, maxUnavailable=0)
RS-v1 [0000]          RS-v2 [ ]          ← start
RS-v1 [0000]          RS-v2 [0]          ← surge new; wait READY
RS-v1 [000 ]          RS-v2 [0]          ← new ready → kill 1 old
RS-v1 [00 ]           RS-v2 [00]         RS-v2 [00]
RS-v1 [ ]             RS-v2 [0000]       ← done (RS-v1 kept @0 for rollback)
rollout undo → scale RS-v1 back up, RS-v2 down (instant)
```

# 21 Real-World Examples

| Scenario                         | Stack & approach                                                                                                                               |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Stateless web API, 3 envs</b> | One Helm chart, values-{dev,staging,prod}.yaml; Deployment + Service + Ingress + HPA; canary via Argo Rollouts                                 |
| <b>Postgres in cluster</b>       | StatefulSet + headless Service + per-pod PVC (volumeClaimTemplates) via a CSI StorageClass; or a Postgres <b>Operator</b> for backups/failover |

| Scenario                | Stack & approach                                                                                           |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Kafka</b>            | StatefulSet (stable broker IDs/identity), per-broker PVC, anti-affinity to spread brokers across nodes/AZs |
| <b>Log collection</b>   | <b>DaemonSet</b> (Fluent Bit) on every node → ship to Elasticsearch/Loki                                   |
| <b>Nightly batch</b>    | <b>CronJob</b> → Job → run-to-completion pod; concurrencyPolicy: Forbid                                    |
| <b>Service mesh</b>     | Istio injects an Envoy <b>sidecar</b> per pod for mTLS, retries, canary traffic-split, tracing             |
| <b>TLS certs</b>        | <b>cert-manager Operator</b> : kind: Certificate → auto-issues + renews from Let's Encrypt                 |
| <b>External secrets</b> | External Secrets Operator syncs from Vault/AWS Secrets Manager into K8s Secrets                            |
| <b>Multi-region</b>     | One cluster per region; ArgoCD ApplicationSet fans the same chart across clusters; global LB routes by geo |

**Cloud managed offerings:** AWS **EKS** (+ Fargate for nodeless pods), GCP **GKE** (Autopilot = fully managed nodes), Azure **AKS**. They run the control plane for you; you bring workloads. AWS **ECS** is a simpler non-K8s orchestrator (good when you don't need K8s's power/complexity).

## 22 Real-Life Analogies

*Shipping and logistics — the whole stack is a port.*

| Concept                         | Analogy                                                                                                                                                                                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Container image</b>          | A standardized shipping container: pack it once, and any ship/truck/crane (any host) handles it identically — that standardization (OCI) is the whole point.                                                                                    |
| <b>Image layers / CoW</b>       | A container packed in labeled strata; to change one stratum you copy just that layer onto a fresh sheet, leaving the shared base pallets untouched for every other container.                                                                   |
| <b>VM vs container</b>          | A VM is a house with its own foundation, plumbing, and power (own kernel) — strong but heavy. A container is an apartment in a shared building (shared kernel) — cheap and dense, but a fire in the shared wiring (kernel) endangers neighbors. |
| <b>Namespaces vs cgroups</b>    | Namespaces are the frosted-glass walls of your apartment (you can only <i>see</i> your own rooms); cgroups are the breaker box that <i>caps</i> how much electricity and water you may draw.                                                    |
| <b>Kubernetes control plane</b> | The port's harbor master: keeps the master manifest (etcd), decides which berth each container goes to (scheduler), and keeps re-checking that reality matches the manifest (controllers).                                                      |
| <b>Reconciliation loop</b>      | A tireless dock foreman who, every minute, counts "manifest says 4 crates, I see 3" and fetches one more — no special "emergency" procedure, just continuous counting and correcting.                                                           |
| <b>Pod</b>                      | A pallet that may carry several boxes (containers) that must travel together, share one address (IP), and be loaded/unloaded as a unit.                                                                                                         |
| <b>Service</b>                  | The port's permanent mailbox number: dockworkers (pods) come and go, but mail addressed to the mailbox always reaches whoever is on duty.                                                                                                       |
| <b>Ingress</b>                  | The port's front gate with a directory: trucks for "shop.example.com/api" are waved to berth A, "/" to berth B, and IDs are checked (TLS) at the gate.                                                                                          |
| <b>Requests vs limits</b>       | Requests = the berth space you reserve in advance (so you're guaranteed a spot); limits = the hard weight cap a crane will lift before it refuses (CPU) or the deck collapses (memory OOM).                                                     |

| Concept              | Analogy                                                                                                                                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Helm chart</b>    | A flat-pack furniture kit: one boxed design (chart) with a parameter sheet (values) — same kit builds the dev desk or the exec desk by filling in different measurements, and a receipt (release/revision) lets you return to the previous build. |
| <b>Helm rollback</b> | The store keeps every past receipt; if the new assembly wobbles, you hand back the receipt and they rebuild the exact previous version.                                                                                                           |
| <b>Operator</b>      | An expert installer who not only assembles the furniture but keeps coming back to tighten bolts, replace worn parts, and upgrade it — your maintenance runbook turned into a permanent on-site worker.                                            |

## 23 Memory Tricks / Mnemonics

**Namespaces (the 7): "Please Name My Unique IP User"** → PID, Net, Mount, UTS, IPC, User, (c)group.

**cgroups vs namespaces: "See vs Use"** → nameSpaces = what you can See; cgroups = what you can Consume.

**CPU vs Memory limit: "CPU is patient, Memory is lethal"** → CPU over limit = **throttle** (slow), Memory over limit = **OOMKill** (dead). *Compressible vs incompressible*.

**QoS order (who dies first): "Best Effort dies Best"** → eviction order **BestEffort** → **Burstable** → **Guaranteed**.

**The three probes: "Start, Live, Ready"** → **Startup** = "done booting?", **Liveness** = "restart if wedged (check self)", **Readiness** = "pull from LB if can't serve (check deps)."

**CMD vs ENTRYPOINT: "ENTRYPOINT is the verb, CMD is the adverb"** → ENTRYPOINT = fixed command, CMD = default args (replaceable).

**Control plane (the 4): "A SAVE C"** → API server, Scheduler, etcd, Controller-manager (+ Cloud-CM).

**Worker node (the 3): "Kube-Kube-Run"** → **kubelet**, **kube-proxy**, **runtime** (containerd).

**Autoscalers: "Pods Out, Pods Up, Nodes Out"** → **HPA** = pods out (more replicas), **VPA** = pods up (bigger), **CA** = nodes out (more machines).

**Helm value precedence: "Set beats File beats Default"** → --set > -f > values.yaml.

**Deployment vs StatefulSet: "Cattle vs Pets"** → Deployment = interchangeable cattle (random names); StatefulSet = named pets with their own beds (stable identity + own PVC + order).

**Docker pipeline: "CLI Drives Containerd Runs"** → docker CLI → **dockerd** → **containerd** → **runc**.

## 24 Common Interview Questions

### 24.1 Q1: What's the difference between Docker and Kubernetes?

**Model answer:** They operate at different layers and aren't alternatives. **Docker** packages and runs a *single* container on *one* host: it builds OCI images, isolates a process with namespaces+cgroups, and runs it via dockerd→containerd→runc. **Kubernetes orchestrates many containers across a fleet of machines:** it schedules pods onto nodes, restarts failed ones, scales replicas, gives them stable networking (Services/DNS), does zero-downtime rolling updates, and manages config/secrets/storage — all driven by declarative desired-state reconciliation. In practice you use Docker (or any OCI tooling) to *build* the image, and Kubernetes to *run* it at scale. Note that K8s doesn't use the Docker daemon at runtime anymore — the kubelet talks CRI directly to containerd.

**Follow-ups:**

- › *Does K8s need Docker?* No. It needs a CRI-compliant runtime (containerd/CRI-O); dockershim was removed in 1.24. Docker is still fine for *building* images.
- › *What does K8s give you that Docker/Compose doesn't?* Multi-node scheduling, self-healing across machines, autoscaling, rolling updates/rollback, Services/Ingress, and declarative reconciliation. Compose is single-host dev only.

## 24.2 Q2: What is a container, really? How does isolation work?

**Model answer:** A container is just a **normal Linux process** with two kernel features applied. **Namespaces** give it an isolated *view* — its own PID space (it's PID 1, can't see host processes), its own network interface and IP, its own filesystem root (the image rootfs via OverlayFS), its own hostname, IPC, and UID mapping. **cgroups** *cap and account* its resource usage — CPU shares/quota and a memory ceiling, so it can't starve neighbors. Optionally **capabilities**, seccomp, and AppArmor trim what even root inside can do. There's no "container" object in the kernel; runc just `clone()`s a process with the right namespace flags, writes cgroup limits, pivots to the image rootfs, and `exec()`s your binary. That's the whole trick — which is also why containers boot in milliseconds (no OS to start) and why isolation is weaker than VMs (one shared kernel).

### Follow-ups:

- › *Namespaces vs cgroups?* Namespaces = what you can **see** (isolation); cgroups = what you can **use** (limits).
- › *Why is a container less isolated than a VM?* Shared host kernel → a kernel exploit = container escape = host compromise. VMs have their own kernel behind a hypervisor. Harden with user namespaces, seccomp, or gVisor/Kata.

## 24.3 Q3: CMD vs ENTRYPOINT --- and exec vs shell form?

**Model answer:** **ENTRYPOINT** is the executable that always runs; **CMD** provides default arguments (or the default command if there's no ENTRYPOINT). At `docker run`, anything you append replaces CMD but is passed as args to ENTRYPOINT. So `ENTRYPOINT ["python", "-m", "app"] + CMD ["--port", "8080"]` runs `python -m app --port 8080`, and `docker run img --port 9090` keeps the entrypoint but swaps the args. Use ENTRYPOINT to fix the binary, CMD for tweakable defaults. The second axis is **form**: *exec form* (JSON array) runs the binary directly as PID 1, so SIGTERM reaches your process and graceful shutdown works; *shell form* (a bare string) wraps it in `/bin/sh -c`, making sh PID 1, which doesn't forward SIGTERM — so Kubernetes ends up SIGKILLing you after the grace period, dropping in-flight requests. Always prefer exec form.

### Follow-ups:

- › *Why does PID 1 matter?* PID 1 has special signal semantics and must reap zombies; if your app isn't getting SIGTERM, use exec form or a tiny init (tini, --init).
- › *How do you override ENTRYPOINT?* `docker run --entrypoint /bin/sh img`.

## 24.4 Q4: Explain multi-stage builds and why they matter.

**Model answer:** A multi-stage Dockerfile has multiple FROM stages; you build in a *fat* stage that has the full toolchain, then `COPY --from=builder` only the final artifact into a *thin* runtime stage, and only the last stage ships. For a Go service, the build stage has the ~1 GB Go toolchain but the runtime stage is a distroless/scratch base holding just the static binary — a single-digit-MB image. Benefits: dramatically smaller images (faster pulls → faster autoscaling and roll-outs, lower storage cost), and a far smaller attack surface (no compilers, no shell, no package manager for an attacker to abuse). Pair it with a non-root USER, a distroless base, and digest-pinning for the senior-grade answer.

### Follow-ups:

- › *distroless vs alpine?* distroless = no shell/pkg-mgr (smallest, hardest to debug — use `kubectl`

debug); alpine = ~5 MB with a shell (musl libc, watch for glibc-incompatibility and DNS quirks).

- › *How do you debug a distroless image?* Ephemeral debug containers (`kubectl debug --image=busybox`) sharing the pod's namespaces.

## 24.5 Q5: Liveness vs readiness probe?

**Model answer:** **Liveness** answers "is this container wedged?" — on failure the kubelet *restarts* the container. **Readiness** answers "can this pod serve traffic right now?" — on failure the pod is *removed from the Service's Endpoints* (gets no traffic) but is **not** restarted, so it can recover on its own. The critical rule: liveness should check only the process itself (a deadlock/hang), while readiness can check dependencies (DB reachable, cache warm). The classic outage is putting a dependency check in the *liveness* probe: when the DB blips, every pod's liveness fails and Kubernetes restarts the entire fleet simultaneously — a restart storm that turns a minor blip into a major outage. There's also a **startup** probe to give slow-booting apps time before liveness/readiness kick in, so an aggressive liveness probe doesn't kill a process that just needs 90 seconds to warm up.

### Follow-ups:

- › *What if you only set a liveness probe?* The pod may receive traffic before it's ready (no readiness gate) and you risk the dependency-restart-storm trap.
- › *Probe types?* httpGet, tcpSocket, exec, grpc — with thresholds and delays.

## 24.6 Q6: Requests vs limits, and the QoS classes?

**Model answer:** **Requests** are what the scheduler *reserves* — it places the pod only on a node with that much free, and requests define the guaranteed floor. **Limits** are the hard cgroup ceiling at runtime. Their relationship sets the pod's **QoS class**, which determines eviction order under node memory pressure: **Guaranteed** (every container has requests == limits) is evicted last; **Burstable** (some requests set, not all equal to limits) is middle; **BestEffort** (nothing set) is evicted first. At runtime the two resources behave differently: exceeding a **CPU** limit only *throttles* you (compressible — you slow down, latency rises, but you keep running), while exceeding a **memory** limit gets you **OOMKilled** (incompressible — exit 137, restart). Best practice: always set requests (so scheduling and QoS work), set memory limits, and be cautious with CPU limits since throttling causes hard-to-diagnose latency. For critical pods, set requests == limits to get Guaranteed.

### Follow-ups:

- › *Why avoid CPU limits sometimes?* CFS throttling can spike p99 latency even when average CPU looks low; many teams set CPU requests but omit CPU limits.
- › *Enforce defaults?* LimitRange + ResourceQuota per namespace so nobody ships BestEffort by accident.

## 24.7 Q7: How does a rolling update work, and how do you roll back?

**Model answer:** A Deployment doesn't mutate pods in place — changing the pod template (e.g., a new image) creates a **new ReplicaSet** and keeps the old one. The Deployment controller then scales the new RS up and the old RS down gradually, governed by `maxSurge` (how many above desired you may temporarily run) and `maxUnavailable` (how many below desired you tolerate). With `maxUnavailable: 0`, `maxSurge: 1` you get true zero-downtime: a new pod is added, and only once its **readiness probe passes** is an old pod removed — repeating until all pods are new. Readiness gating means a broken new version stalls the rollout instead of taking down the old pods. **Rollback** is instant because the old ReplicaSet (with its full pod template) is retained: `kubectl rollout undo` just scales the previous RS back up and the current one down. `kubectl rollout status/history` track it.

**Follow-ups:**

- ▶ *What if the new version is bad but passes readiness?* Use canary/blue-green or Argo Rollouts with automated metric analysis and auto-rollback.
- ▶ *maxSurge vs maxUnavailable trade-off?* maxUnavailable:0 needs spare capacity for the surge pod; higher maxUnavailable rolls faster but reduces capacity mid-roll.

**24.8 Q8: StatefulSet vs Deployment --- when and why?**

**Model answer:** Use a **Deployment** for stateless apps where pods are interchangeable "cattle" — random names, no stable identity, shared/no storage, all updated at once. Use a **StatefulSet** for stateful apps (databases, Kafka, ZooKeeper) that need: **stable network identity** (ordinal names db-0/1/2 with per-pod DNS via a headless Service), **ordered operations** (db-0 Ready before db-1 starts; reverse to scale down), and **per-pod persistent storage** (each pod gets its *own* PVC via `volumeClaimTemplates`, re-attached to the same ordinal on reschedule). These matter because clustered databases do leader election, replication, and membership based on stable identity, and each replica owns distinct data — random names and shared volumes would break replication and corrupt data.

**Follow-ups:**

- ▶ *Why a headless Service?* It gives each pod a stable DNS A-record (db-0.db.ns.svc) so peers can address specific members, instead of a load-balanced VIP.
- ▶ *What sticks the storage to a pod?* The PVC created from `volumeClaimTemplates` is named per-ordinal and re-bound when that ordinal reschedules.

**24.9 Q9: How does an external request reach a pod?**

**Model answer:** DNS resolves the hostname to a **cloud load balancer** (provisioned by a Service type=LoadBalancer fronting the ingress controller). The LB forwards to a **NodePort** on the worker nodes; on that node, **kube-proxy**'s iptables/IPVS rules DNAT the traffic to an **ingress-controller pod** (e.g., nginx). The ingress controller terminates TLS and matches host/path rules to pick the right backend **Service**. It (or kube-proxy again) then DNATs the Service's ClusterIP to one healthy backing **pod IP**, chosen from the live Endpoints list that the Endpoints controller keeps current via the Service's label selector. Finally the **CNI** routes the packet to that pod — a local veth if same-node, or an overlay/BGP route if remote. Every hop is config-driven and self-updating as pods churn, which is the whole reason Services exist: pods are mortal and change IPs, but the Service VIP and DNS name are stable.

**Follow-ups:**

- ▶ *What does kube-proxy actually do?* In iptables/IPVS mode it *programs kernel rules*; the kernel forwards packets — kube-proxy isn't in the data path.
- ▶ *Why Ingress over many LoadBalancer Services?* One LB + L7 host/path routing + TLS for many apps, instead of one expensive cloud LB per service.

**24.10 Q10: How does the scheduler decide where to place a pod?**

**Model answer:** For each unscheduled pod, kube-scheduler runs two phases. **Filter (predicates):** eliminate nodes that *cannot* run it — not enough allocatable CPU/memory for the pod's requests, nodeSelector/required affinity mismatch, untolerated taints, volume zone/topology conflicts, node not Ready or cordoned, hostPort conflicts. **Score (priorities):** rank the surviving feasible nodes 0–100 with weighted functions — spread vs bin-pack (LeastAllocated/MostAllocated), balanced cpu:mem usage, affinity/anti-affinity and topology-spread preferences, and image locality (a node that already cached the image scores higher for faster startup). It picks the highest-scoring node and **binds** by writing `nodeName` back via the API server; the kubelet on that node then actually starts the pod. If no node survives filtering, the pod stays **Pending**, which is what triggers the Cluster Autoscaler to add a node.

**Follow-ups:**

- › *Taints vs affinity?* Affinity = pods *attract* to nodes; taints = nodes *repel* pods unless tolerated (effects NoSchedule/PreferNoSchedule/NoExecute).
- › *Spread replicas across zones?* podAntiAffinity or topologySpreadConstraints with topologyKey: topology.kubernetes.io/zone.

**24.11 Q11: What problem does Helm solve, and Helm vs Kustomize?**

**Model answer:** Helm is the Kubernetes package manager; it solves **YAML sprawl and release management**. A real app is a dozen interrelated manifests that must vary per environment — hand-copying them causes drift and gives no atomic deploy/rollback. Helm packages them into one **chart** with Go-templated manifests parameterized by a **values** file, so the same chart serves dev/staging/prod (and is redistributable to other users). It also tracks each install as a versioned **release**, enabling `helm upgrade`, `helm rollback` to a prior revision, `--atomic` auto-rollback on failed upgrades, and lifecycle hooks (e.g., pre-upgrade migrations). **Kustomize** takes a different approach: no templating language — you keep plain-YAML *bases* and apply per-environment *overlay patches* (`kubectl apply -k`). Choose **Helm** when you need packaging/distribution, heavy parameterization, and release lifecycle (rollback/hooks); choose **Kustomize** when you own all the YAML and want simple, template-free env variation. Many teams combine them — Helm for third-party charts, Kustomize for in-house services, or even `helm template | kubectl apply -k`.

**Follow-ups:**

- › *Helm 2 vs 3?* Helm 3 removed **Tiller**, the privileged in-cluster server that was a security hole (it could deploy anything as cluster-admin); Helm 3 is a thin client using *your* RBAC, and stores releases as namespaced Secrets.
- › *Values precedence?* `--set` overrides `-f` files which override the chart's `values.yaml`.

**24.12 Q12: Walk me through debugging a CrashLoopBackOff pod.**

**Model answer:** CrashLoopBackOff isn't an error type — it means the container keeps starting and exiting, so the kubelet restarts it with exponential backoff. My job is to find *why it exits*. First, `kubectl describe pod <p>` to read the Events and the container's last State/Reason, especially the **exit code**: 137 means OOMKilled (raise the memory limit or fix a leak), 1/2 usually means an application error (bad config, missing env/Secret, failed DB connection), 126/127 means the command isn't executable/found (bad ENTRYPOINT). Then the key command, `kubectl logs <p> --previous`, shows the *crashed* container's output — the actual stack trace or panic. Common root causes: a missing ConfigMap/Secret or wrong env var, a failed migration, or an overly aggressive liveness probe killing a slow-booting app (fix with a startup probe). If the container won't stay up long enough to inspect, I'll override the command to `sleep` or attach an ephemeral debug container with `kubectl debug` to get a shell and reproduce. I'd also `kubectl get events --sort-by=.lastTimestamp` for surrounding context like node pressure or image issues.

**Follow-ups:**

- › *Exit code 137 specifically?* 128+SIGKILL(9) = killed — usually OOMKilled (check `describe` "Reason: OOMKilled") or a failed liveness probe.
- › *It's stuck ImagePullBackOff instead?* Different branch: bad image name/tag, missing `imagePullSecret` for a private registry, registry auth/rate-limit, or node can't reach the registry — `describe` shows the exact pull error.

**25 Senior-Level Discussion Points**

## 25.1 Why Kubernetes is "just" a database of controllers

The deepest framing: K8s is fundamentally **etcd + an API server + a set of controllers running level-triggered reconciliation loops**. The API server is a glorified, validated, watchable database; controllers are clients that watch for diffs and act. This is why it's so extensible — a CRD adds a table, an Operator adds a controller, and your custom thing behaves exactly like built-ins. *Level-triggered* (converge to desired state) rather than *edge-triggered* (react to events) is the key robustness property: a controller can crash, restart, miss events, and still converge correctly because it re-derives desired-vs-actual from scratch each loop. Saying this signals you understand the architecture's *essence*, not just its API.

## 25.2 The liveness-probe footgun and cascading failures

A liveness probe that checks a downstream dependency turns a transient dependency blip into a self-inflicted fleet-wide restart storm: DB hiccups → all pods fail liveness → all restart at once → connection storm to the recovering DB → worse outage. The principle: **liveness checks the process's own health; readiness checks ability to serve**. This generalizes to a senior insight — *health checks should fail in a way that sheds load gracefully (readiness/remove from LB), not in a way that amplifies the failure (mass restart)*.

## 25.3 Stateful workloads are the hard part

K8s nailed stateless scale-out; stateful is genuinely harder. StatefulSets give identity/ordering/per-pod storage, but real databases need bootstrapping, leader election, backup, point-in-time restore, and version upgrades — which is exactly why **Operators** exist (encode the DBA's runbook as a controller). The senior take: "run stateful on K8s only with a mature operator, and even then weigh it against a managed cloud DB (RDS/Cloud SQL) — the operational burden of self-hosting Postgres on K8s is real."

## 25.4 CPU limits considered (sometimes) harmful

A non-obvious production lesson: setting CPU *limits* can hurt latency because CFS throttling kicks in at period boundaries even when average utilization is modest, causing p99 spikes. Many high-scale shops set CPU *requests* (for scheduling/QoS) but deliberately omit CPU *limits*, while always setting memory limits (since memory is incompressible and a leak can take down a node). Knowing this trade-off — and that it's contested — is a strong signal.

## 25.5 Multi-tenancy and the shared-kernel trust boundary

Because containers share the host kernel, true hostile multi-tenancy (running untrusted code from many customers) on plain containers is risky — one kernel CVE = escape. Senior answer: use **node-per-tenant isolation, gVisor/Kata** (sandboxed runtimes), strong **NetworkPolicy + PSA/Kyverno**, and per-workload ServiceAccounts with minimal RBAC. This is the security reasoning behind serverless platforms' architecture.

## 25.6 GitOps as the deployment control plane

The mature deployment story isn't `kubectl apply` from CI — it's **GitOps** (ArgoCD/Flux): Git is the source of truth, a controller continuously reconciles the cluster to match the repo. This composes perfectly with K8s's reconciliation model (it's reconciliation all the way up), gives auditability (git log = deploy log), easy rollback (git revert), and drift detection. Helm/Kustomize render the manifests; ArgoCD applies and self-heals them.

## 25.7 When NOT to use Kubernetes

Senior judgment includes knowing the cost. K8s has enormous operational complexity; for a small team or simple app, **ECS/Fargate, Cloud Run, App Runner, or a PaaS** deliver containers

with a fraction of the cognitive load. Adopt K8s when you genuinely need its portability, ecosystem (operators, mesh), multi-cloud leverage, or scale — not because it's fashionable. "Use the least powerful tool that solves the problem" plays well.

## 26 Typical Mistakes Candidates Make

| Mistake                                       | Correction                                                                                                                                    |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| "Docker and Kubernetes are competitors"       | Different layers: Docker builds/runs one container; K8s orchestrates many. K8s uses containerd, not the Docker daemon, at runtime.            |
| "A container is a lightweight VM"             | A container is a <i>process</i> with namespaces+cgroups sharing the host kernel — no guest OS. VMs virtualize hardware with their own kernel. |
| Confusing liveness and readiness              | Liveness restarts (check self); readiness removes from LB (check deps). Dependency checks in liveness cause restart storms.                   |
| "Secrets are encrypted"                       | Secrets are <b>base64-encoded</b> , not encrypted, by default. Enable etcd encryption-at-rest + tight RBAC or use an external manager.        |
| Setting only limits, no requests              | Without requests you get BestEffort QoS (first evicted) and bad scheduling. Always set requests.                                              |
| "CPU over limit kills the pod"                | CPU over limit <i>throttles</i> (compressible). <b>Memory</b> over limit OOMKills (incompressible).                                           |
| Using :latest tags                            | Mutable and non-reproducible; you can't tell what's deployed and can't reliably roll back. Pin tags/digests.                                  |
| Shell-form ENTRYPOINT                         | sh -c becomes PID 1 and doesn't forward SIGTERM → ungraceful shutdown/SIGKILL. Use exec form (JSON array).                                    |
| "StatefulSet is just Deployment with storage" | It's identity + ordering + <i>per-pod</i> PVC + headless DNS — needed for clustered databases' membership/replication.                        |
| Treating Helm as just templating              | It's also <b>release management</b> : versioned revisions, rollback, atomic upgrades, hooks. That's the half Kustomize lacks.                 |
| "Helm needs Tiller"                           | Helm 3 removed Tiller. Saying "Tiller" reveals you're on outdated knowledge — and Tiller was a security hole.                                 |
| Putting all replicas on one node/zone         | No fault tolerance. Use podAntiAffinity / topologySpread across nodes and AZs.                                                                |
| Forgetting that pods are ephemeral            | Pod IPs/names change; never hardcode them. Use Services/DNS. Data must go to a PVC, not the writable layer.                                   |
| Giving workloads broad RBAC / default SA      | Per-workload ServiceAccount with least privilege. A compromised pod with cluster-admin = game over.                                           |
| Ignoring DNS at scale                         | CoreDNS + ndots:5 is a top scale failure mode. Mention NodeLocal DNSCache / FQDNs.                                                            |

## 27 How This Connects to Other Topics

### 27.1 Distributed Systems (07-distributed-systems.md)

- › **etcd uses Raft** — the consensus algorithm from that file; it's CP, needs majority quorum, and is the cluster's single source of truth.
- › K8s Services + CoreDNS = **service discovery**; kube-proxy = **server-side load balancing**.
- › Pods are mortal and IPs churn — the **8 fallacies** (topology changes, network unreliable)

made manifest; Services/Ingress exist to abstract them.

- ▶ Rolling updates, leader election in operators, and split-brain avoidance in StatefulSets all reuse distributed-systems primitives.

## 27.2 Cloud & Infrastructure (04-cloud-infrastructure.md)

- ▶ This file is the deep dive *under* that one. Service mesh (Istio/Envoy sidecars), CI/CD, GitOps (ArgoCD), Terraform-provisioned EKS/GKE, and Prometheus/Grafana observability all sit on top of the K8s described here.
- ▶ Deployment strategies (canary/blue-green/rolling) connect to that file's progressive-delivery section.

## 27.3 System Design

- ▶ "Design X" answers almost always run **stateless services as Deployments behind Services/Ingress, autoscaled with HPA**, data in StatefulSets or managed DBs, async work via Jobs/queues. Containers are the assumed substrate.
- ▶ Bin-packing, autoscaling, and multi-AZ spread are capacity/availability design levers.

## 27.4 Operating Systems / Linux

- ▶ Namespaces, cgroups, capabilities, OverlayFS, seccomp/AppArmor are **Linux kernel features** — containers are an OS topic at heart. PID 1 signal semantics tie to process/signal fundamentals.

## 27.5 Networking

- ▶ CNI flat network, iptables/IPVS DNAT, overlays (VXLAN), L7 routing (Ingress), TLS termination, NetworkPolicy firewalls — all applied networking concepts.

## 27.6 Security

- ▶ Supply chain (image scanning/signing/SBOM), least privilege (non-root, drop caps, RBAC), secrets management, admission control (Kyverno/Gatekeeper), and multi-tenant isolation (gVisor/Kata).

# 28 FAANG Interview Tips

**Lead with the mental model, then the mechanism.** For any K8s question, anchor on "declarative desired state + reconciliation loops through one API server, etcd as source of truth." For containers, anchor on "a process with namespaces (what it sees) + cgroups (what it uses)." Then go deep on the specific mechanism.

**Always trace the failure path.** Interviewers reward "here's what happens when X breaks": node dies → ReplicaSet recreates; memory exceeds limit → OOMKill 137 → restart → CrashLoop; bad liveness probe → restart storm. Failure-mode fluency separates senior candidates.

**Name the trade-off explicitly.** "CPU limit throttles vs memory limit OOM-kills, so I set memory limits but am cautious with CPU limits." "Helm gives release lifecycle; Kustomize gives template-free simplicity." "StatefulSet for identity, but consider a managed DB to avoid the operational burden."

**Tie to the company.** Amazon → ECS/EKS/Fargate, cost, operational excellence. Google → scheduler internals, etcd, Borg lineage, SRE/SLO. Netflix → canary with automated analysis, chaos. Apple → security, supply chain.

**Quantify when you can.** "maxUnavailable:0 means I need spare capacity for the surge pod

but get true zero-downtime.” HPA:  $\text{ceil}(\text{currentReplicas} \times \text{current}/\text{target metric})$ .” distroless takes the image from ~1 GB to single-digit MB.”

**Know what's deprecated/modern.** dockershim removed (1.24) → CRI/containerd. Tiller removed → Helm 3. PodSecurityPolicy removed → Pod Security Admission / Kyverno. Ingress → Gateway API direction. Cluster Autoscaler → Karpenter on AWS. Saying the current state signals you're hands-on.

**Red flags to avoid:** calling a container a lightweight VM; saying Secrets are encrypted; mentioning Tiller as current; putting dependency checks in liveness; using `:latest`; claiming Docker and K8s compete; not knowing requests vs limits.

**For the Helm question specifically:** don't stop at "it templates YAML." Say "**templating + release management**," name values precedence (`--set > -f > defaults`), name rollback/`--atomic`/hooks, and contrast with Kustomize (overlays, built into `kubectl`, no releases). That completeness is exactly the gap this topic tests.

## 29 Revision Cheat Sheet

### 29.1 10-Minute Summary

1. **Container** = Linux process + **namespaces** (what you see) + **cgroups** (what you use) + capabilities. Shares host kernel. No guest OS. ms boot, MB image.
2. **Docker pipeline:** CLI → dockerd → containerd → runc (sets up ns/cgroups, exec's your process, exits). K8s skips dockerd via **CRI** → **containerd**.
3. **Images** = stacked read-only layers + writable layer, merged by OverlayFS, copy-on-write. Order Dockerfile least→most-changed for cache. **Multi-stage** = build fat, ship thin (distroless + non-root). Pin by **digest**.
4. **CMD vs ENTRYPOINT:** ENTRYPOINT = fixed binary, CMD = default args. **Exec form** (JSON) → PID 1 gets SIGTERM (graceful); shell form → `sh` eats the signal.
5. **K8s control plane:** apiserver (only door + RBAC + admission), **etcd** (Raft, source of truth), scheduler, controller-manager, CCM. **Nodes:** kubelet, kube-proxy, runtime.
6. **Reconciliation loop:** declare desired state → controllers level-trigger actual→desired forever. Self-healing is just the loop noticing a diff.
7. **Workloads:** Pod (pause container holds netns; init + sidecar) · Deployment (rolling update via new ReplicaSet; rollback = re-scale old RS) · **StatefulSet** (stable ID + order + per-pod PVC) · DaemonSet (one/node) · Job/CronJob (run-to-completion/scheduled).
8. **Networking:** flat IP-per-pod, **no NAT**. Service (ClusterIP/NodePort/LoadBalancer/ExternalName/Headless) = stable VIP via label selector; kube-proxy DNAT (iptables vs **IPVS**). CoreDNS discovery. **Ingress** = L7 host/path/TLS behind one LB. NetworkPolicy = firewall.
9. **Config:** ConfigMap (plain) vs **Secret (base64, NOT encrypted!)** → enable etcd encryption-at-rest. Storage: PVC (claim) → PV (disk) → StorageClass (dynamic provision) → CSI. RWO vs RWX.
10. **Scheduling:** filter (can it fit? selectors/taints/affinity) → score → bind. **Requests** = scheduler reservation; **limits** = cgroup cap. **QoS:** Guaranteed > Burstable > BestEffort (eviction order). **CPU over limit = throttle; memory over limit = OOMKill**.
11. **Autoscaling:** HPA = more pods ( $\text{ceil}(\text{replicas} \times \text{cur}/\text{target})$ ); VPA = bigger pods; CA/Karpenter = more nodes. Combo: HPA + CA.
12. **Probes:** startup (booting?) · liveness (restart if wedged — check **self**) · readiness (pull from LB if can't serve — check **deps**). Dep-check in liveness = restart storm.
13. **Helm:** package manager = **templating + release management**. Chart (Chart.yaml/values.yaml/templating) · Go templates: `.Values`, `.Release`, `include`, `toYaml` | `nindent`. Releases/revisions → `upgrade/rollback/--atomic/hooks`. Precedence: `--set > -f > values.yaml`. Helm 3 killed **Tiller** (security). **vs Kustomize** = templating+lifecycle vs overlays-in-kubectl.

14. **Debugging:** describe (events/state/exit code) → logs --previous → get events. CrashLoop = why it exits (137=OOM); ImagePullBackOff = name/tag/secret/registry; Pending = scheduling (requests/taints/affinity/PVC).

## 29.2 Key Comparison Tables (memorize)

### Containers vs VMs:

Container: shared kernel, ns+cgroups, MB, ms, weak isolation, 100s/host  
 VM: own kernel, hypervisor, GB, sec, strong isolation, 10s/host

### Workload picker:

Deployment → stateless cattle (rolling update, random names)  
 StatefulSet → stateful pets (stable ID + order + per-pod PVC) → DBs/Kafka  
 DaemonSet → one per node (log/metrics/CNI agents)  
 Job/CronJob → run-to-completion / scheduled batch

### Resource behavior:

CPU > limit → THROTTLE (compressible, slow, alive)  
 Memory > limit → OOMKILL (incompressible, exit 137, restart)  
 QoS eviction order: BestEffort → Burstable → Guaranteed

### Probes:

startup → grace for slow boot (gates the others)  
 liveness → restart container (check SELF)  
 readiness → remove from Service Endpoints (check DEPENDENCIES)

### Autoscalers:

HPA → #replicas | VPA → pod size | CA/Karpenter → #nodes

### Helm vs Kustomize:

Helm: templating + values + RELEASES (rollback/hooks), charts/repos  
 Kustomize: overlay PATCHES on a base, no templates, built into kubectl (-k)  
 Precedence (Helm): --set > -f file > values.yaml

### Service types:

ClusterIP → internal VIP (default)  
 NodePort → static port on every node (dev/on-prem)  
 LoadBalancer → cloud LB + external IP (prod)  
 ExternalName → CNAME to external host  
 Headless → no VIP, returns pod IPs (StatefulSet DNS)

### 29.3 Most Important Concepts (Priority Order)

1. Container = process + namespaces + cgroups (what you see / what you use)
2. Declarative reconciliation loops + control-plane/node components
3. Requests vs limits + QoS + throttle-vs-OOMKill
4. Liveness vs readiness vs startup (and the restart-storm trap)
5. Deployment rolling update + rollback (new RS / old RS)
6. StatefulSet vs Deployment (identity/order/per-pod PVC)
7. Service/Ingress/kube-proxy/CoreDNS — external→pod path
8. Helm = templating + release management; vs Kustomize; Helm 3 no Tiller
9. Scheduler filter→score→bind; taints/affinity
10. Debugging CrashLoop/ImagePull/Pending (describe → logs --previous → events)

*Study tip: For every object, be able to state (1) what problem it solves, (2) the mechanism (what the kernel/controller actually does), (3) the YAML shape, (4) the trade-off, and (5) a failure scenario. That five-part structure answers nearly any containers/K8s/Helm question an interviewer can throw.*